

Beyond Backpropagation: Exploring Innovative Algorithms for Energy-Efficient Deep Neural Network Training

Przemyslaw Spyra
przspyra@student.agh.edu.pl

AGH University of Science and Technology
Krakow, Poland

Abstract

The increasing computational demands of deep neural networks (DNNs), primarily driven by the high-energy requirements of the backpropagation (BP) algorithm, pose a significant challenge to sustainable AI development. This paper presents a rigorous investigation into three promising training methods without backpropagation: the **Forward-Forward (FF) Algorithm**, the **Cascaded-Forward (CaFo) Algorithm**, and the **Mono-Forward (MF) Algorithm**. This investigation charts an evolutionary progression from foundational concepts to a demonstrably superior solution.

To ensure a fair and insightful analysis, a robust comparative framework was established. Each alternative algorithm was implemented on its native architecture (MLPs for FF and MF, a CNN for CaFo) and systematically compared against an equivalent model trained with BP. Critically, this methodology mandates comprehensive hyperparameter optimization for all algorithms using Optuna, coupled with consistent early stopping criteria based on validation performance, thereby ensuring that all comparisons are conducted between optimally tuned models.

The investigation culminates in a significant finding: The Mono-Forward (MF) algorithm not only competes with, but consistently surpasses backpropagation in classification accuracy on its native MLP architectures. This superior generalization arises from the algorithm's ability to converge to a more favorable minimum in the validation loss landscape, challenging the long-held assumption that global optimization is a prerequisite for achieving state-of-the-art performance. This breakthrough delivers profound efficiency gains, reducing energy consumption by up to 41% and accelerating training by up to 34% on complex datasets. These figures represent direct measurements at the hardware level obtained via the NVIDIA Management Library (NVML) API, which translate to a demonstrably smaller carbon footprint as estimated by the CodeCarbon tool.

Beyond this primary result, this paper provides a detailed analysis of the underlying drivers of computational efficiency. This "autopsy at the hardware level" reveals the sources of performance disparities by exposing the architectural inefficiencies of the FF algorithm while validating the computationally lean design of MF. Furthermore, this research empirically challenges the simplistic notion that all BP-free methods are inherently more memory-efficient, instead revealing the nuanced resource requirements of each approach. By documenting the trajectory from FF's foundational concepts and their practical limitations to MF's compelling synthesis of superior accuracy and sustainability, this work offers a clear, data-driven roadmap for the future of energy-efficient deep learning.

1 Introduction

1.1 Motivation

Deep neural networks (DNNs) have become a transformative technology in machine learning and artificial intelligence, establishing state-of-the-art benchmarks in diverse domains such as computer vision [30, 19], natural language processing [47, 14], and reinforcement learning [34]. However, this success is predicated on a substantial and escalating demand for computational resources, which translates directly into significant energy consumption. The challenge is particularly acute, as contemporary models now routinely scale to billions or trillions of parameters. The predominant training paradigm, which relies on the backpropagation (BP) algorithm [42], which requires the storage of all intermediate activations during the forward-pass to facilitate gradient computation in the subsequent backward pass. This requirement results in considerable memory usage and a correspondingly high energy budget [10]. Moreover, the inherently sequential nature of the forward and backward passes imposes a fundamental constraint on parallelization, a bottleneck known as "backward locking" [26].

The environmental consequences of these computational demands have emerged as a pressing concern. Recent analyses reveal a paradox in modern AI, where its capacity to optimize energy systems is contrasted with its own operational requirements. Although specific projections vary according to scope and underlying assumptions, some forecast that AI could account for **up to 3% of global electricity demand by 2030** [48], underscoring an undisputed trend of rapid escalating energy consumption. Architectural studies confirm that even modest reductions in network complexity can halve the energy consumed during training [41], while hardware-aware analyzes underscore the non-linear relationship between model scale and energy cost [13]. In particular, training a single large language model can produce a carbon footprint of **more than 500 metric tons of CO₂**, a figure equivalent to the lifetime emissions of several conventional automobiles [39, 1]. In response, a new generation of hardware accelerators, sparsity techniques, and analytical tools such as CodeCarbon [12], which provides an estimated CO₂e from energy use and local grid intensity, is being developed to tackle this energy crisis.

This context has spurred a search for alternative training algorithms that completely eliminate the backward pass. Among the most promising are Geoffrey Hinton’s Forward-Forward (FF) algorithm [21], the Cascaded-Forward (CaFo) algorithm [52], and the Mono-Forward (MF) algorithm [16]. These backpropagation-free (BP-free) methods are not only intended to reduce energy consumption but are also inspired by learning principles observed in biological neural systems. In contrast to backpropagation’s dependence on a global error signal and non-local computations, these alternatives employ local, layer-wise learning mechanisms. Such mechanisms are analogous to synaptic plasticity in the brain, where learning is governed by localized phenomena such as Hebbian learning [44] and spike-timing dependent plasticity (STDP) [8].

By aligning more closely with these neurobiological principles, these alternative algorithms offer potential improvements in energy efficiency and scalability. They can reduce computational overhead and memory requirements and, by breaking the global dependency of the backward pass, create new opportunities for parallel and distributed processing across diverse architectures, including multi-layer perceptrons (MLPs) and convolutional neural networks (CNNs). This localized learning paradigm could be especially advantageous in resource-constrained environments, such as on edge devices or within Internet of Things (IoT) systems. Ultimately, the biological inspiration that underpins these methods fosters the potential to develop more robust and efficient learning strategies, bridging the gap between artificial and natural intelligence and paving the way for a new generation of deep learning algorithms that are powerful and sustainable [6, 32].

1.2 Problem Statement

The conventional backpropagation (BP) algorithm, despite its widespread success, is constrained by inherent limitations. Currently, the emerging class of BP-free alternatives introduces its own set of unresolved challenges related to practical viability and comparative performance. First, standard BP is constrained by significant drawbacks, including high memory costs for activation storage, the substantial computational overhead of the backward pass, the backward locking effect that inhibits parallelism, and the potential for gradient instability [6, 32, 26, 22]. Second, there is a distinct lack of systematic studies that rigorously compare emerging BP-free algorithms such as FF, CaFo, and MF against BP under controlled conditions, particularly those that evaluate both performance and direct energy efficiency metrics [27, 45].

This gap is exacerbated by a third issue: Architectural variations frequently confound existing comparisons. Studies often assess alternative algorithms on architectures that are distinct from standard BP configurations, making it difficult to disentangle the influence of the training algorithm from that of the architectural design [13, 41]. A critical deficiency is the absence of comparisons where alternative algorithms are evaluated on their specific "native" architectures, and fair backpropagation baselines are constructed on those same architectures, a condition essential for an accurate assessment. Fourth, energy efficiency analysis is often inconsistent, relying on indirect proxies such as floating-point operations (FLOPs) rather than direct hardware monitoring. This practice can yield incomplete or misleading conclusions about the true energy profiles of different training methods [40, 9, 11], and standardized reporting of estimated CO₂e using tools such as CodeCarbon is often omitted. Finally, as a consequence of these issues, the specific trade-offs between classification performance and a comprehensive suite of efficiency metrics (energy, time, memory, GFLOPs, estimated CO₂e) for these newer algorithms relative to established baselines of BP remain inadequately characterized, thus limiting their informed adoption [41, 35].

Addressing these deficiencies is imperative for the advancement of sustainable deep learning. Therefore, a standardized methodology is required to rigorously evaluate the performance and efficiency of emerging training algorithms against established baselines on equitable architectural grounds.

1.3 Research Objectives and Questions

To address this need, the primary objective of this paper is to conduct a comprehensive and fair comparison of the Forward-Forward (FF), Cascaded-Forward (CaFo), and Mono-Forward (MF) algorithms against a standard backpropagation-based approach. The experimental design focuses on implementing each algorithm in its native network architecture and establishing a corresponding fair BP baseline that uses an identical architecture, thereby isolating the impact of the training method itself.

This paper seeks to provide empirical answers to the following research questions:

1. How do the alternative training algorithms (FF, CaFo, and MF) compare to backpropagation in terms of classification accuracy and convergence rate on the MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100 datasets? This investigation is structured to evaluate FF foundationally on Fashion-MNIST, CaFo in its native CNN architecture, and MF in its native MLP architecture across all relevant datasets.
2. What are the directly measured energy consumption profiles (via the NVIDIA Management Library (NVML)) of FF, CaFo, and MF compared to their fair backpropagation baselines during training on identical native architectures?
3. How do key computational efficiency metrics differ between the alternative algorithms and their fair BP baselines under controlled architectural settings, including MLP structures native to MF?

This analysis covers:

- Wall-clock training time.
 - Peak GPU memory usage.
 - Estimated forward-pass Giga Floating-Point Operations (GFLOPs) as a proxy for inference complexity.
 - An estimation of the computational cost per update cycle for BP, illustrating the theoretical savings from eliminating the backward pass.
 - Estimated Carbon Dioxide Equivalent (CO₂e) calculated using the CodeCarbon library.
4. What specific energy performance trade-offs (including estimated CO₂e) exist for CaFo and MF relative to backpropagation? How does FF’s trade-off compare within its more limited experimental scope?
 5. How does the scalability, in terms of both performance and efficiency, of CaFo (on CNNs) and MF (on MLPs) vary across datasets of increasing complexity when compared to BP, and what are the implications for their potential deployment in energy-constrained environments?

1.4 Paper Contributions

This paper makes several distinct contributions to the field of energy-efficient deep learning:

- **Rigorous Algorithm Replication:** The author faithfully reproduced the native architectures for the FF (MLP), CaFo (CNN, including both Rand CE and the implemented DFA CE variants), and MF (MLP) algorithms from their original descriptions.
- **Fair Comparative Framework:** By implementing both alternative algorithms and their backpropagation counterparts on identical network architectures and applying consistent practices such as early stopping based on validation performance, this paper establishes a framework that isolates the impact of the training algorithm itself.
- **Empirical Energy Efficiency Analysis:** Through direct hardware measurements using the NVIDIA NVML API and profiling tools, this work provides a detailed, multifaceted efficiency profile for each training method under fair conditions. This profile includes direct energy, time, peak memory, estimated forward-pass GFLOPs, and an estimated CO₂e, alongside an estimation of BP’s per-update computational cost to highlight algorithmic differences.
- **Trade-off Characterization:** This paper presents a thorough analysis of the trade-offs between classification performance, determined from the best model state identified by validation, and comprehensive energy efficiency, offering valuable insights for the practical deployment of deep learning models in energy-conscious settings.
- **Scalability Assessment:** Through systematic experiments on datasets of increasing complexity (MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100), the scalability and practical applicability of these novel training methods are evaluated.
- **Open Research Infrastructure:** To promote reproducibility and facilitate future research, all source code, structured configuration files (employing an inheritance-based YAML system), experimental protocols and implementations of measurement tools are publicly available in a GitHub repository, with additional details provided in the Appendix C.

1.5 Overview of Paper Structure

This paper is organized into six chapters, each designed to build on the preceding one to construct a comprehensive investigation of energy-efficient training methods for DNNs:

- **Chapter 1: Introduction** outlines the motivation, defines the problem statement, and details the research objectives, contributions, and general structure of the paper.
- **Chapter 2: Background and Literature Review** provides an overview of deep neural networks and the backpropagation algorithm, reviews pressing challenges in energy efficiency, and details the alternative training algorithms (FF, CaFo, MF) alongside the rationale for fair benchmarking. This chapter frames the selected algorithms within an evolutionary context, beginning with FF’s foundational concepts, progressing to CaFo’s structured block-wise methodology, and culminating in MF’s recent advancements in both efficiency and performance.
- **Chapter 3: Methodology** describes the experimental design, dataset preprocessing, implementation details for the alternative algorithms and their corresponding fair backpropagation baselines, evaluation metrics, and hardware and software infrastructure. A key focus of this chapter is the establishment of a methodologically rigorous framework for ensuring fair comparisons.
- **Chapter 4: Experiments and Results** presents a detailed account of the empirical findings, including a full suite of performance and efficiency metrics for both the alternative algorithms and their fair BP baselines, supported by comparative analyses and statistical evaluations.
- **Chapter 5: Discussion** interprets the experimental results, examines the specific trade-offs between energy efficiency and model performance, and discusses the advantages, limitations, and practical implications of the investigated training methods.
- **Chapter 6: Conclusion and Future Work** summarizes the key findings, revisits the contributions of the paper, acknowledges its limitations, and proposes promising directions for future inquiry.

This structure is intended to guide the reader systematically through the context, methods, results, and implications of this research, which explores alternatives beyond conventional backpropagation to foster a more energy-efficient future for deep learning.

2 Literature Review

This chapter establishes the theoretical foundations and contextual framework essential for interpreting the comparative analysis presented in this paper. It commences with an exposition of deep neural networks and the canonical backpropagation algorithm, before proceeding to a discussion of the energy efficiency challenges endemic to deep learning. The chapter subsequently provides a detailed examination of three alternative training paradigms: the Forward-Forward (FF), Cascaded-Forward (CaFo), and Mono-Forward (MF) algorithms, which constitute the central focus of this research. These algorithms are situated within an evolutionary narrative of BP-free learning, where each method either builds upon or diverges from preceding concepts to resolve specific limitations. The chapter culminates by exploring established benchmarking practices for ensuring fair comparison against backpropagation and identifying the specific gaps in the existing literature that this paper seeks to address, with a pronounced emphasis on methodological rigor.

2.1 Deep Neural Networks and Backpropagation

Deep neural networks (DNNs) have become established as a class of powerful models capable of discerning complex patterns from large-scale datasets, thereby facilitating breakthrough performance across a multitude of domains [17]. Fundamentally, DNNs are constructed from multiple layers of interconnected computational units, or neurons, which transform input data through a sequence of non-linear mappings to produce semantically meaningful outputs. The efficacy of these networks is largely attributable to their hierarchical architecture, which enables the learning of progressively more abstract data representations.

The canonical training methodology for DNNs is the backpropagation (BP) algorithm [42], a technique that has retained its dominance for over three decades. Backpropagation operates via two distinct phases: a forward pass, during which input signals traverse the network to generate predictions, and a backward pass, where error gradients are propagated from the output layer to preceding layers to guide the adjustment of weights. This process is formally defined as follows.

For a neural network comprising L layers, let $\mathbf{x}_l$ denote the activations in layer l , $\mathbf{W}_l$ the weight matrix connecting layer $l - 1$ to layer l , $\mathbf{b}_l$ the bias vector in layer l and f_l the corresponding activation function. The forward pass computes:

$$\mathbf{z}_l = \mathbf{W}_l \mathbf{x}_{l-1} + \mathbf{b}_l \quad (2.1)$$

$$\mathbf{x}_l = f_l(\mathbf{z}_l) \quad (2.2)$$

In the subsequent backward pass, for a given loss function $\mathcal{L}$ that measures the discrepancy between the predictions of the network $\mathbf{x}_L$ and the target outputs $\mathbf{y}$, the gradients are computed. The process begins at the output layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \odot f'_L(\mathbf{z}_L) \quad (2.3)$$

For any hidden layer l (where $1 \leq l < L$), the gradient is propagated backward:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = \left(\mathbf{W}_{l+1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}} \right) \odot f'_l(\mathbf{z}_l) \quad (2.4)$$

Finally, the gradient with respect to the weights of the layer l is calculated as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} \mathbf{x}_{l-1}^T. \quad (2.5)$$

As detailed in Equations (2.1) and (2.2), the forward pass entails the computation of pre-activation values followed by the application of activation functions. The backward pass, initiated by Equation (2.3) in the output layer, propagates the gradient through each hidden layer via the chain rule, as shown in Equation (2.4). Equation (2.5) then specifies how the gradients with respect to the weights are determined.

Despite its demonstrated efficacy, backpropagation is beset by several inherent limitations. First, it requires the storage of all intermediate activations from the forward pass for subsequent use in the backward pass, a requirement that leads to substantial memory consumption scaling linearly with network depth [10]. This consequently increases the energy expenditure associated with data storage and movement. Second, the algorithm’s sequential nature gives rise to a computational bottleneck known as backward locking, wherein weight updates for a given layer are deferred until the completion of both the forward and backward passes, including gradient computations from all subsequent layers. This dependency obstructs layer-parallel updates and reduces opportunities for holistic training parallelization [26]. Third, the algorithm is susceptible to the *vanishing or exploding gradients* problem, where backpropagated error signals either diminish exponentially, rendering updates to early layers ineffective, or grow uncontrollably, destabilizing the training process. This phenomenon is primarily caused by the repeated multiplication of Jacobian factors (related to f'_l and $\mathbf{W}_{l+1}^T$ in Equation (2.4)) through the deep network chain [22].

In addition, backpropagation has faced criticism for its **lack of biological plausibility**. The algorithm relies on precise, symmetric weight transport between forward and backward passes (i.e., the use of $\mathbf{W}_{l+1}^T$ in Eq. 2.4) is a requirement for which there is no substantial evidence in biological neural circuits [32]. The propagation of global error signals also stands in contrast to the predominantly local learning mechanisms observed in biological neurons [6].

These constraints have catalyzed research into alternative training methodologies that seek to ameliorate the deficiencies of backpropagation while preserving competitive levels of performance. Of particular relevance are approaches that obviate the backward pass entirely, offering potential enhancements in memory efficiency, training velocity, and energy consumption.

2.2 Energy Efficiency in Deep Learning

The exponential scaling of computational demands for training contemporary deep neural networks has raised significant concerns about energy consumption and environmental impact. Seminal studies have shown that the training of a single large language model can generate carbon emissions comparable to the lifetime emissions of five automobiles [45]. This phenomenon is symptomatic of a wider increase in the energy requirements of information and communication technologies (ICT), with reports indicating that data centers and communication networks are now responsible for 2 to 3% of global electricity consumption and approximately 1% of greenhouse gas emissions [49]. Although initial projections from 2015 suggested that ICT could constitute a significant portion of global electricity usage by 2030 [3], more recent analyses from the Oxford Institute for Energy Studies forecast a base case scenario where data centers alone will consume 1,135 TWh in the same year [37]. The combined electricity demand from data centers, cryptocurrencies and AI is expected to increase substantially by 2026 [49], highlighting the need for energy-efficient deep learning paradigms.

The energy consumption profile of deep learning is multifaceted. A primary contributor is the computational workload of the training process itself, particularly matrix multiplications and evaluations of activation functions, which require considerable computational power. Floating-point operations (FLOPs) are commonly employed as an abstract metric to estimate these computational costs.

However, it is crucial to recognize that, while FLOPs provide a useful measure of computational complexity, they do not directly quantify energy consumption. FLOPs represent a simple tally of arithmetic operations, whereas the actual energy expenditure per operation can exhibit

significant variance contingent on hardware specifics such as processor architecture, numerical precision, and power state. Furthermore, a substantial fraction of energy, often rivaling or exceeding that of computation, is consumed by data movement across the memory hierarchy (e.g., DRAM, cache, registers) and interconnects, a factor that correlates poorly with FLOP counts [23].

Moreover, the notion of a fixed FLOP count for a fundamental operation is itself being challenged by AI-driven algorithmic discovery. For example, DeepMind AlphaTensor, a deep reinforcement learning agent, discovered novel matrix multiplication algorithms that are asymptotically more efficient than established methods like Strassen’s algorithm for certain matrix sizes [51]. By identifying a method to multiply 4x4 matrices in 47 steps instead of Strassen’s 49, it achieved the first such improvement in more than fifty years, demonstrating that the foundational number of operations for a task is not an immutable baseline. This development serves as a powerful parallel to the objective of this paper: just as AI can optimize a core *operation* such as matrix multiplication, this research investigates the replacement of the core training *algorithm*, backpropagation, to achieve greater efficiency. Ultimately, the dynamic nature of algorithmic efficiency underscores the limitations of relying solely on theoretical proxies and reinforces the methodological decision in this work to rely on direct, empirical measurements of energy, time, and memory for a robust and faithful comparison.

This need for a nuanced and empirical approach is especially clear when evaluating backpropagation (BP). An analysis restricted to forward-pass FLOPs is insufficient, as a complete BP update cycle encompasses a forward-pass, a backward-pass for gradient computation, and subsequent weight updates. As a widely accepted heuristic, the backward pass typically entails approximately twice the computational workload (FLOPs) of the forward pass for standard network layers [28]. Consequently, the total theoretical computational cost of a single BP update cycle, which this paper terms F_{BP_update} , can be estimated relative to the forward pass cost (F_{fwd}) as follows:

$$F_{BP_update} \approx F_{fwd}(\text{forward}) + 2 \times F_{fwd}(\text{backward}) = 3 \times F_{fwd} \quad (1)$$

BP-free methods, by their design, eliminate the $2 \times F_{fwd}$ cost of the backward pass. Although this 3x estimate serves as a useful theoretical benchmark, its ultimate impact on overall training efficiency is modulated by factors such as convergence speed and hardware utilization, reinforcing the need for direct time and energy measurements. In this paper, both the estimated forward pass GFLOPs (F_{fwd}) for all algorithms and the estimated BP update cycle GFLOPs (F_{BP_update}) are reported to illustrate this computational difference.

More direct and accurate measurements of energy consumption can be achieved through specialized APIs and instrumentation. The NVIDIA Management Library (NVML) API, for instance, provides real-time power readings from GPU devices, facilitating precise energy usage estimates during training. Tools such as CarbonTracker [4] and CodeCarbon [12] utilize these measurements, integrating factors such as runtime and regional carbon intensity of the electricity grid to estimate the associated carbon footprint, typically reported in Carbon Dioxide Equivalent (CO2e) [12].

Despite the availability of these tools, systematic reporting of energy metrics remains inconsistent within the deep learning research community [20]. Many studies prioritize model accuracy and convergence speed, relegating energy considerations to a secondary status. When energy metrics are reported, they frequently lack methodological standardization, which complicates direct comparisons between different approaches.

The concept of "Green AI" [43] has arisen in response to these challenges, advocating for the elevation of efficiency to a primary evaluation criterion along with precision. This paradigm encourages the development of models that achieve state-of-the-art performance while minimizing resource consumption, thereby moving away from a paradigm of pursuing marginal accuracy gains with ever larger models at a significant environmental cost.

It is also pertinent to note that many existing techniques for mitigating the energy cost of deep learning, such as network pruning, quantization [25], knowledge distillation, and the development of inherently efficient architectures like MobileNets [24] and EfficientNets [46], primarily target the model’s structure or its inference phase. Although these methods are valuable, they are largely *orthogonal* to the choice of the core training algorithm that dictates how weights are updated. This paper is specifically focused on the potential energy savings that can be achieved by fundamentally altering this training algorithm and moving beyond backpropagation.

Alternative training algorithms that eliminate or modify the backpropagation process represent a promising avenue to improve energy efficiency during the *training* phase. By potentially reducing memory overhead, enabling superior parallelization, and minimizing the data movement and computation associated with the backward pass, these approaches have the potential to substantially decrease the energy footprint of deep learning training while maintaining competitive performance.

2.3 Alternative Training Algorithms: An Evolutionary Perspective

This section investigates three innovative training algorithms: Forward-Forward (FF), Cascaded-Forward (CaFo), and Mono-Forward (MF), which are designed to transcend the limitations of backpropagation. These methods are presented as stages in an evolutionary progression towards BP-free learning. FF established the fundamental concept of local, forward-only training. CaFo advanced this idea by introducing a structured block-wise framework to integrate supervisory signals more directly, although with its own set of complexities. MF represents a more recent refinement, developed to achieve high performance and efficiency through a distinct local learning mechanism. A common characteristic shared by all three algorithms is the elimination of the global backward pass; they rely instead on local learning rules that update weights based solely on information available within each layer.

2.3.1 Forward-Forward (FF) Algorithm

Proposed by Geoffrey Hinton in 2022, the FF algorithm [21] constitutes a radical departure from conventional backpropagation, motivated by considerations of biological plausibility and the potential for greater computational efficiency. Its central innovation was to demonstrate that effective learning can be achieved through purely local objectives and two distinct forward passes: one for "positive" (real) data and another for "negative" (typically artificially generated) data, thereby obviating the need to propagate error gradients backwards. Each layer is trained locally to differentiate between these data types by optimizing a layer-specific "goodness" metric [21].

The core mechanism of the algorithm is as follows:

1. **Data Presentation:** In a supervised context, positive data is formulated by presenting an input, such as an image, paired with its correct label. This pairing is often accomplished by embedding the label directly into the input representation, for example, by replacing the first few pixels. Subsequently, negative data is created by pairing the same input with an incorrect label [21].
2. **Goodness Function:** Each layer l calculates a goodness score derived from its activation pattern $\mathbf{x}_l$. A common formulation for this score is the sum of squared activations: $G_l(\mathbf{x}_l) = \sum_i x_{l,i}^2$. The weights of the layer are then adjusted to increase this goodness for positive samples while decreasing it for negative samples [21].
3. **Local Objective:** The learning objective for a layer with parameters θ_l is designed to maximize the difference in goodness between positive ($\mathbf{x}_l^+$) and negative ($\mathbf{x}_l^-$) inputs.

This is frequently framed using a logistic loss function applied to this difference, which may be shifted by a threshold θ :

$$\mathcal{L}_l(\boldsymbol{\theta}_l) = \log(1 + e^{-(G_l(\mathbf{x}_l^+) - \theta)}) + \log(1 + e^{(G_l(\mathbf{x}_l^-) - \theta)}) \quad (2.6)$$

where the optimization goal is to minimize $\mathcal{L}_l$.

4. **Layer Normalization:** Before propagating activations $\mathbf{x}_l$ to the subsequent layer $l + 1$, FF applies layer normalization [5], which is often simplified to length normalization by dividing by the L2 norm. This step is of critical importance: it removes the magnitude information that defined the goodness in layer l , thereby compelling subsequent layers to learn new discriminative features based on the orientation or relative activities of the activation vector, rather than merely inheriting the goodness signal [21].

This forward-only, local learning paradigm presents several potential advantages pertinent to energy-efficient training:

- **Reduced Memory Footprint:** By eliminating the backward pass, FF obviates the need to store intermediate activations for gradient computation. Memory requirements scale with the size of the layer rather than the depth of the network, which offers a significant potential for savings relative to BP, particularly in deep networks [21].
- **Enhanced Parallelizability:** As each layer updates its weights locally and independently without backward locking, FF has the potential to facilitate greater parallelism across layers or even enable pipelined processing of data streams [21].
- **Avoidance of Gradient Issues:** The local nature of the objective function inherently bypasses the issues of vanishing or exploding gradients associated with deep backpropagation chains [21].

However, FF also presents a unique set of characteristics and challenges:

- **Native Architecture (MLP Focus):** It is essential to note that Hinton’s original research mainly focused on developing and evaluating FF on MLPs with fully-connected layers, such as four-layer networks with 2000 ReLU neurons each for MNIST and Fashion-MNIST [21]. Hinton himself noted difficulties in adapting FF to CNNs. This paper adheres to the native MLP architecture for FF evaluation, specifically a 4×2000 network for Fashion-MNIST as per the research plan.
- **Inference Procedure:** The standard inference process in FF requires multiple forward passes. To classify a given input, the network computes the total goodness, summed across relevant layers, for the input paired with each possible class label. The label that produces the highest total goodness is then selected as the prediction [21].
- **Performance Relative to Backpropagation:** The results of Hinton’s work and subsequent studies generally indicate that FF, in its native MLP form, achieves reasonable accuracy on datasets like MNIST but tends to lag behind standard BP, particularly on more complex datasets such as CIFAR-10 [21]. Convergence is also typically observed to be slower [21].
- **Hyperparameter Sensitivity:** The algorithm can exhibit sensitivity to hyperparameters such as learning rates and goodness thresholds (θ in Eq. 2.6), and the specifics of the normalization procedure [21].

From a biological point of view, FF is considered as more plausible than BP because it employs local learning rules, avoids the need for precise symmetric weight transport, and does not require long-term storage of activations [21].

In the context of this paper, FF serves as a foundational example of a BP-free algorithm, representing a critical conceptual advance. Its potential for reduced memory usage and enhanced parallelizability makes it highly relevant to the study of energy efficiency, although its practical efficiency in the tests conducted for this paper proved to be limited. By evaluating FF on its native MLP architecture against an identical MLP trained with BP, this research aims to isolate the impact of the FF mechanism on both performance and energy metrics.

2.3.2 Cascaded-Forward (CaFo) Algorithm

The CaFo algorithm [52], proposed by Zhao et al. in 2023, builds upon concepts introduced in FF and aims to address some of the limitations of Hinton’s approach, including its reliance on negative samples and the indirect nature of the "goodness" signal for supervision. CaFo introduces a structured, block-wise training process designed to improve stability, convergence, and accuracy while retaining the advantages of BP-free learning.

CaFo is characterized by several key features:

1. **Cascaded Neural Blocks:** The architecture comprises sequential "neural blocks," each containing standard components such as convolutional or fully-connected layers, normalization, and activation functions that progressively extract features [52].
2. **Layer-wise Predictors (Auxiliary Classifiers):** A dedicated predictor, typically a shallow fully-connected layer followed by a softmax function, is attached to each neural block. This predictor maps the block’s output features directly to task-specific outputs, such as class probabilities, thereby providing a more direct supervisory signal at multiple depths within the network [52].
3. **Independent Component Training:** The predictors can be trained independently, often using the outputs of pre-trained or even randomly initialized neural blocks. The blocks themselves can be trained using non-BP methods such as direct feedback alignment (DFA) [36] or remain fixed [52].
4. **Single Input Stream:** In contrast to the positive/negative passes of FF, CaFo utilizes a single input stream and employs ground truth labels for local predictor losses, which simplifies data handling [52].
5. **Knowledge Integration during Inference:** The final prediction is typically derived by aggregating the outputs from all predictors, for example, by averaging their probability distributions. This approach utilizes information from multiple levels of representation in a single forward pass through the architecture [52].

Native Architecture and Implementation for Fair Benchmarking: As specified in the CaFo publication [52], and pertinent to the methodology of this paper, the authors proposed and evaluated a specific CNN variant for image classification tasks. This native architecture, which was used for datasets including Fashion-MNIST and CIFAR-10/100, involves several cascaded convolutional blocks. For the experiments in this paper, this research adheres strictly to this native CNN architecture. It consists of **three blocks**, each containing a **Conv2D(3x3)**, **ReLU**, **MaxPool(2x2)**, and **BatchNorm** layer, with progressively increasing channel depths (e.g., **32, 128, 512 channels**). The output feature map of each block is flattened before being fed into its dedicated predictor (e.g., an **FC layer + Softmax**). The corresponding fair BP baseline replicates this exact CNN architecture and is trained end-to-end.

Training Methodology and Efficiency Aspects: CaFo training involves distinct phases, which have been implemented as part of this work:

- **Neural Block Training (Optional: CaFo-DFA Variant):** The neural blocks can be randomly initialized and kept fixed, which constitutes the *CaFo-Rand* variant. Alternatively, they can be trained using a BP-free method such as direct feedback alignment (DFA) [36]. DFA offers a solution to the biological implausibility of backpropagation by eliminating the need for symmetric weight transport. Instead of propagating the error backward using the transpose of the forward weight matrices, DFA generates a synthetic gradient for each layer. This is achieved by projecting the global error signal from the output layer directly back to each hidden layer via a fixed, random feedback matrix. As this matrix is not learned and is independent of the forward weights, it allows each layer to compute an approximate weight update locally, thus avoiding the backward locking problem. Although often yielding lower accuracy than BP, its mechanism serves as an important benchmark for other BP-free algorithms [16, Table 3]. This DFA block training mechanism has been implemented, which uses an approximation of DFA for convolutional layers, as a configurable option for evaluating the *CaFo-DFA* variant.
- **Predictor Training:** Each predictor is trained independently on the output of its corresponding block. CaFo supports multiple loss functions, including Mean Squared Error (MSE), Cross-Entropy (CE), and Sparsemax Loss (SL) [52]. It should be noted that using the MSE loss for predictors permits a **closed-form solution** via linear regression, allowing exceptionally fast predictor training once the block outputs ($\mathbf{H}$) are computed [52]. Although the MSE loss variant offers this exceptional training speed, this work focuses on the Cross Entropy (CE) variants, as they are reported to yield higher classification accuracy, providing a more stringent test against backpropagation’s performance [52]. Training with CE or SL involves gradient descent but is based solely on local gradients computed through the shallow predictor.

This modular training approach, which eliminates the deep backward pass, suggests potential advantages in terms of reduced memory usage and faster training times, particularly when using MSE loss, making it highly relevant for energy efficiency analysis. The use of DFA for block training offers an additional BP-free pathway within the CaFo framework.

Performance and Trade-offs: The results presented in [52] indicate that CaFo significantly outperforms FF on benchmarks such as MNIST and CIFAR-10/100. Although it still generally underperforms an end-to-end BP-trained model on the same architectures, the performance gap reported for CaFo, especially with CE or SL loss and DFA-trained blocks, is smaller than that of FF [52]. The choice of loss function (MSE vs. CE/SL) presents a trade-off between speed and accuracy [52]. The experiments in this paper are designed to quantify these performance and efficiency trade-offs against a fair BP baseline, with a primary focus on the Rand-CE variant, while also acknowledging the implemented DFA capability.

Biological Plausibility: Similar to FF, CaFo is argued to be more biologically plausible than BP due to its avoidance of weight transport, achieved through the use of DFA or local predictors, and its lack of global error propagation [52].

In summary, CaFo represents an evolutionary step from FF, offering a structured forward-only learning approach with the potential for improved accuracy and specific efficiency advantages. Its distinct native architectures and training options, including the Rand-CE and DFA-CE variants implemented, establish it as a key algorithm for the comparative study in this paper.

2.3.3 Mono-Forward (MF) Algorithm

The MF algorithm, introduced by Gong, Li, and Abdulla [16], is a recent BP-free training method that builds on the core principle of local, forward-only learning. It employs a novel mechanism, local projection matrices, with the objective of achieving performance that can match or even surpass that of backpropagation, particularly on MLP architectures, while offering substantial benefits in memory efficiency and parallelizability. MF simplifies certain aspects of both FF, by eliminating the need for negative samples, and CaFo, by replacing complex cascaded predictor training with a direct local loss computation on each layer’s activations.

Core Mechanism: Local Learning with Projection Matrices The key innovation of MF lies in its layer-wise optimization process, which utilizes a dedicated *projection matrix* $\mathbf{M}_i$ in each hidden layer i . This matrix, with dimensions $m \times n$ (where m is the number of classes and n is the number of neurons in the layer), maps the layer’s activation vector $\mathbf{a}_i$ (of size $1 \times n$ for a single sample, or $B \times n$ for a batch of size B) directly to category-specific "goodness" scores $\mathbf{G}_i$ (of size $1 \times m$ or $B \times m$):

$$\mathbf{G}_i = \mathbf{a}_i \mathbf{M}_i^T \quad (2.7)$$

MF then applies a standard cross-entropy loss, computed locally at each layer i , between the softmax of these goodness scores and the true labels $\mathbf{y}$:

$$\mathcal{L}_i = \text{CrossEntropy}(\text{softmax}(\mathbf{G}_i), \mathbf{y}) \quad (2.8)$$

Critically, both the layer’s weights $\mathbf{W}_i$ and its projection matrix $\mathbf{M}_i$ are updated using gradients derived exclusively from this local loss $\mathcal{L}_i$:

$$\mathbf{W}_i \leftarrow \mathbf{W}_i - \eta \frac{\partial \mathcal{L}_i}{\partial \mathbf{W}_i} \quad (2.9)$$

$$\mathbf{M}_i \leftarrow \mathbf{M}_i - \eta \frac{\partial \mathcal{L}_i}{\partial \mathbf{M}_i} \quad (2.10)$$

where η represents the learning rate. The gradient for the projection matrix, $\frac{\partial \mathcal{L}_i}{\partial \mathbf{M}_i}$, is computed directly. The gradient for the layer weights is determined via a local "mini-backpropagation" through the layer itself, as defined by the chain rule:

$$\frac{\partial \mathcal{L}_i}{\partial \mathbf{W}_i} = \frac{\partial \mathcal{L}_i}{\partial \mathbf{G}_i} \frac{\partial \mathbf{G}_i}{\partial \mathbf{a}_i} \frac{\partial \mathbf{a}_i}{\partial \mathbf{z}_i} \frac{\partial \mathbf{z}_i}{\partial \mathbf{W}_i} \quad (2.11)$$

where $\mathbf{z}_i$ is the pre-activation value. This local mechanism effectively avoids backward locking and the problems associated with global gradient propagation [16].

Native Architectures and Fair Benchmarking Relevance The original MF paper [16] primarily evaluated the algorithm using MLPs on standard image classification benchmarks. The native architectures specified in the paper, which are relevant to this paper, are:

- **MNIST & Fashion-MNIST:** A 2-hidden-layer MLP with 1000 ReLU neurons per hidden layer (a **2×1000 ReLU MLP**).
- **CIFAR-10 and CIFAR-100:** A 3-hidden-layer MLP with 2000 ReLU neurons per hidden layer (a **3×2000 ReLU MLP**).

Each hidden layer in these architectures incorporates its own projection matrix $\mathbf{M}_i$, which is initialized using a method such as Kaiming uniform initialization [18], and is trained using the local loss $\mathcal{L}_i$. In the methodology of this paper, these specific MLPs will be implemented for MF, and the fair BP baselines will utilize *identical* MLP structures trained end-to-end.

Prediction Methods MF supports two distinct approaches for inference [16]:

1. **FF-style Aggregation:** The goodness scores ($\mathbf{G}_i$) from all layers are aggregated, for instance, by summation, and the class corresponding to the highest aggregate score is predicted.
2. **BP-style Final Layer:** Only the goodness scores from the final hidden layer ($\mathbf{G}_L$) are used for prediction via a softmax function. This method is computationally analogous to standard BP inference, requiring a single forward pass.

The original paper suggests that BP-style inference is both efficient and performs well [16].

Potential for Energy Efficiency The design of the MF algorithm holds significant promise for energy efficiency:

- **Memory Usage:** The elimination of the backward pass obviates the need to store intermediate activations. The MF paper provides empirical evidence of substantially lower peak memory usage compared to BP [16, Fig. 1-3].
- **Training Time:** The layer-wise independence of MF removes the backward locking constraint, which enables the potential layer-parallelism. The paper reports that MF trains significantly faster per epoch than BP on comparable MLP architectures [16, Table 5].
- **FLOPs:** While MF eliminates the FLOPs associated with the backward pass, it introduces additional FLOPs for projection matrix computations. The net effect on computational load requires empirical measurement.
- **Energy Consumption:** The combination of lower memory requirements, potentially faster training, and parallelizable computations is expected to result in a reduced overall energy consumption. Direct measurements using NVML are necessary to verify this.

It should be noted that the projection matrices $\mathbf{M}_i$ add to the parameter count compared to a standard BP network, leading to a slight increase in model size [16].

Performance and Convergence Crucially, Gong et al. [16] report that MF consistently **matched or surpassed the accuracy of BP** on identical MLP architectures across the MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100 datasets, while also significantly outperforming FF, FA, and DFA [16, Table 3]. MF also demonstrated remarkable convergence stability [16, Fig. 5].

Biological Plausibility and Modularity MF aligns more closely with the principles of biological learning than BP, due to its use of local updates and the absence of weight transport [16]. The independence of its layers also confers a high degree of modularity [16, Section 3.3].

In summary, MF represents a significant recent advancement in the field of BP-free learning. It offers a compelling alternative that has the potential to deliver superior performance along with substantial efficiency advantages by directly mapping local layer activations to class-specific scores for local loss computation. The evaluation of MF on its native MLP architectures will be a primary focus of this paper’s comparative analysis against fair BP baselines.

2.4 Benchmarking Against Backpropagation and the Need for Fairness

A rigorous evaluation of the performance and energy efficiency of alternative training algorithms relative to backpropagation requires a fair benchmarking methodology. However, a review of the literature reveals that comparisons are often confounded by several factors. The most prominent of these is the network architecture; discrepancies in layer types, depth, width, connectivity, and activation functions can exert a dramatic influence on both performance and energy consumption, independent of the training algorithm used [13, 41].

Previous studies comparing alternative algorithms often lacked the strict consistency required for a truly equitable assessment. For instance, comparisons frequently use standard BP architectures that may not replicate the potentially non-standard native architectures of the alternative algorithms, thereby hindering a fair assessment of the algorithmic impact. Furthermore, best practices such as systematic hyperparameter tuning and consistent validation-based early stopping are not always universally applied across all compared methods [21], making it difficult to disentangle algorithmic effects from those specific to a particular experimental setup. Research focused on energy efficiency has also highlighted the sensitivity of such measurements to architectural and configuration details [15], further underscoring the necessity for the architecturally fair and methodologically rigorous comparison that this paper employs. Addressing these issues requires a framework built upon several principles of fairness:

- **Architectural Parity:** The alternative algorithm and its backpropagation baseline should be evaluated on identical network structures.
- **Systematic Optimization:** All the compared methods should undergo a rigorous, equivalent hyperparameter tuning to ensure that each performs at its potential.
- **Consistent Evaluation Criteria:** Practices like early stopping based on validation performance must be universally applied to avoid penalizing algorithms with different convergence dynamics.
- **Comprehensive and Consistent Metrics:** Efficiency should be evaluated using direct hardware measurements rather than relying solely on proxies.

The absence of a standardized methodology that incorporates these principles represents a significant gap in the literature. By adopting a fair benchmarking approach, which is centered on native architectures and consistent training practices, this paper aims to provide a robust assessment of the true performance and energy trade-offs offered by FF, CaFo, and MF relative to standard backpropagation.

2.5 Summary of Related Work and Identification of Research Gaps

This chapter has reviewed the foundational principles of DNN training with backpropagation, the significant challenge of energy efficiency in deep learning, and three promising BP-free alternatives: the Forward-Forward (FF), Cascaded-Forward (CaFo), and Mono-Forward (MF) algorithms, framed within an evolutionary context. Key conclusions drawn from the literature are as follows:

1. Backpropagation, despite its dominance, is characterized by limitations related to memory consumption, parallelism due to backward locking, gradient stability, and biological plausibility [32, 6, 26].
2. The energy cost associated with training large-scale DNNs is substantial and continues to grow, requiring research into more efficient training methodologies [45, 39, 49].
3. Alternative algorithms such as FF [21], CaFo [52], and MF [16] offer potential solutions by eliminating the backward pass and utilizing local learning rules.
4. These alternatives are often developed and evaluated on specific "native" architectures, such as MLPs for FF and MF or block-wise CNNs for CaFo, a factor critical for fair assessment [21, 52, 16].

Despite this progress, several research gaps persist, providing the motivation for this paper:

1. **Absence of a Fair and Holistic Comparative Framework:** There is a scarcity of studies that systematically compare emerging BP-free algorithms against backpropagation under truly equitable conditions. A rigorous framework would require implementing algorithms on their respective native architectures, establishing architecturally identical BP baselines, and applying consistent best practices such as universal hyperparameter optimization and validation-based early stopping to all methods.
2. **Inconsistent and Indirect Efficiency Measurement:** Many studies lack direct hardware-level energy measurements, often relying on proxies like FLOPs which may not reflect true energy profiles [9]. The systematic and transparent reporting of a comprehensive suite of metrics, including direct energy consumption (Wh), training time, peak memory, and estimated CO₂e, is rarely performed for these newer algorithms against fairly-optimized baselines [20].
3. **Uncharacterized Performance-Efficiency Trade-offs:** As a consequence of the above, the specific trade-offs between classification accuracy and a full suite of measured efficiency metrics remain inadequately characterized for algorithms such as CaFo and MF when compared to rigorously-optimized BP baselines [35, 41].
4. **Unknown Scalability on Native Architectures:** It is not well understood how the performance and efficiency of algorithms like CaFo and MF, on their native architectures, scale across datasets of increasing complexity when compared to fair and optimized BP baselines.

This paper addresses these gaps by conducting a rigorous comparative empirical study. It focuses on the performance and, critically, the energy efficiency of FF, CaFo, and MF against fair BP baselines implemented on their identical native architectures, where all methods benefit from systematic hyperparameter tuning and early stopping. The methodology detailed in Chapter 3 is designed to facilitate this fair comparison through direct hardware measurements and computational profiling.

3 Methodology

This chapter details the research methodology used to investigate the research questions articulated in Chapter 1. The author outlines the experimental design, the datasets utilized, the specific implementations of the alternative algorithms on their native architectures, and the corresponding configurations for fair BP baselines. Furthermore, this chapter describes the systematic hyperparameter optimization strategy, the metrics for performance and efficiency evaluation, and the computational infrastructure. The central tenet of this methodology is the establishment of a rigorous and equitable comparative framework, achieved by controlling for architectural variations and applying consistent optimization and evaluation protocols across all experiments.

3.1 Ensuring Fair Comparisons: Methodological Rigor

A cornerstone of this paper is the adherence to a strict methodological protocol designed to ensure that all comparisons between the alternative algorithms and backpropagation (BP) are equitable. To address the need for fair benchmarking identified in the literature (see Section 2.4), this rigor allows the conclusion that observed differences are primarily due to the training algorithms themselves, rather than to confounding variables. To this end, the author implemented the following principles:

1. **Native Architecture Replication:** Each alternative algorithm was implemented in its specific native architecture, as described or evaluated in its source publication. This principle respects the developmental context of each algorithm. Consequently, FF and MF were evaluated on their designated MLP structures, while CaFo was evaluated on its defined CNN with three blocks for image classification tasks.
2. **Identical Architectures for Backpropagation Baselines:** For each experiment involving an alternative algorithm, a corresponding BP baseline was constructed using an *identical* network architecture. This entailed replicating the number and type of layers, neuron and channel counts, activation functions, and any integrated normalization layers. Algorithm-specific components, such as FF’s goodness layers or MF’s projection matrices, were excluded from the BP baseline, which instead featured a standard final classification layer suitable for end-to-end training. This critical step isolates the impact of the training algorithm from that of the architectural design.
3. **Systematic and Universal Hyperparameter Optimization:** To mitigate the influence of default parameters and allow each algorithm to operate near its optimal capacity, systematic hyperparameter optimization was applied using the Optuna framework. This tuning was performed universally for *all* alternative algorithms and their respective BP baselines, for every experimental configuration. This practice ensures that comparisons are not biased by disparities in default tuning.
4. **Consistent Early Stopping Based on Validation Performance:** All training procedures incorporated early stopping, governed by performance on a separate validation set. Training was terminated if the validation score failed to improve over a predefined patience period. The model checkpoint that produced the best validation score was subsequently used for final evaluation on the test set and efficiency measurements. This protocol ensures that the results reflect the optimal state achieved by each model and avoids comparisons based on arbitrary training durations, which could unfairly bias algorithms with different convergence dynamics.

5. **Consistent Software and Hardware Environment:** All experiments were executed within a unified software environment and on identical hardware (NVIDIA A100 GPUs), thus minimizing the variability due to software versions or hardware idiosyncrasies.
6. **Standardized Data Handling and Metrics:** Identical data preprocessing, augmentation, and dataset splits were employed for all compared algorithms within a given task. All performance and efficiency metrics were measured using consistent tools and methodologies in each experiment.

By adhering to these principles, this research provides a robust and transparent evaluation, facilitating reliable conclusions regarding the relative merits and trade-offs of the investigated BP-free training algorithms.

3.2 Experimental Design Overview

The research is based on a comparative experimental design devised by the author to isolate the effect of the training algorithm on both task performance and computational efficiency. The design encompasses the following key comparisons:

- **Forward-Forward (FF) vs. Backpropagation (BP):** A foundational set of experiments was carried out on MLP architectures. This involved implementing FF and a fair BP baseline on a 4×2000 ReLU MLP for the Fashion-MNIST dataset, and on both 3×1000 and 4×2000 ReLU MLPs for the MNIST dataset. These architectures are consistent with those explored in the original FF publication [21] and serve as a reference for BP-free methods.
- **Cascaded-Forward (CaFo) vs. Backpropagation (BP):** A comprehensive evaluation was performed on the MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100 datasets. CaFo was implemented on its native CNN architecture with three blocks, as specified for image datasets in its source paper [52]. The evaluation encompassed both the **CaFo-Rand-CE** and **CaFo-DFA-CE** variants. Comparisons were made against fair BP baselines utilizing the identical CNN structure with three blocks.
- **Mono-Forward (MF) vs. Backpropagation (BP):** An extensive evaluation of the MF algorithm was performed across the four datasets using its specified native MLP architectures [16]. For the MNIST and Fashion-MNIST datasets, a **2×1000 ReLU MLP** was employed. For the more complex CIFAR-10 and CIFAR-100 datasets, a larger **3×2000 ReLU MLP** was used. In all instances, MF was benchmarked against fair BP baselines constructed with these identical MLP structures.

In every comparison, the principle of using identical architectures for both the alternative algorithm and its corresponding BP baseline was strictly maintained, as detailed in Section 3.1. Furthermore, all training procedures integrated early stopping based on the performance of the validation set. This involved monitoring a specified metric and terminating training if no improvement was observed over a predefined patience period. The final evaluation metrics were reported from the model checkpoint corresponding to the best validation score achieved, a standard practice that ensures that the results reflect the optimal state reached by each model. Although the original publications for FF, CaFo, and MF may have utilized different library versions or fixed training durations, the author conducted all experiments within a consistent, modern software environment (detailed in Section 3.8) and used early stopping to determine the effective training duration, thus ensuring an equitable comparison of algorithms on contemporary hardware.

3.3 Dataset Description and Preprocessing

The following standard benchmark datasets were utilized to evaluate the algorithms across varying levels of classification complexity:

- **MNIST [31]:** A dataset of 70,000 28x28 pixel grayscale images of handwritten digits (0 to 9), with a standard 60k/10k train/test split.
- **Fashion-MNIST [50]:** A dataset of 70,000 28x28 pixel grayscale images across 10 categories of apparel, with a standard 60k/10k train/test split.
- **CIFAR-10 [29]:** A dataset of 60,000 32x32 pixel color images spanning 10 object classes, with a standard 50k/10k train/test split.
- **CIFAR-100 [29]:** A dataset similar to CIFAR-10 but with 100 distinct classes, also using a standard 50k/10k train/test split.

For each dataset, the standard training and testing splits were preserved. For CIFAR-10, CIFAR-100, and Fashion-MNIST, a validation subset was created by randomly sampling 10% of the original training data. Following the precedent established in the original FF paper [21], a fixed split was used for the MNIST dataset, allocating 50,000 images for training and 10,000 for validation. The remainder of the original training set was used for model training in all cases. In adherence to recommended practices for computational workloads on the Athena cluster, datasets were loaded from and intermediate results were stored on the user’s high-performance \$SCRATCH directory on the Lustre filesystem.

Consistent preprocessing steps were applied across all experiments for any given dataset:

- **Normalization:** Pixel values were first scaled to a range via `ToTensor`. Subsequently, the images were normalized using the standard means and standard deviations per channel for each dataset:
 - **MNIST:** mean=(0.1307,), std=(0.3081,)
 - **Fashion-MNIST:** mean=(0.2860,), std=(0.3530,)
 - **CIFAR-10:** mean=(0.4914, 0.4822, 0.4465), std=(0.2023, 0.1994, 0.2010)
 - **CIFAR-100:** mean=(0.5071, 0.4867, 0.4408), std=(0.2675, 0.2565, 0.2761)
- **Data Augmentation (CIFAR):** For the CIFAR-10 and CIFAR-100 datasets, standard data augmentation techniques, specifically random horizontal flips ($p=0.5$) and random crops (32x32 with padding of 4 pixels), were applied consistently during the training of both alternative algorithms and their BP baselines.
- **Data Augmentation (MNIST/Fashion-MNIST):** In accordance with common practice, no data augmentation was applied to the MNIST or Fashion-MNIST datasets.
- **Dimensions:** Input images were processed at their native resolutions. These were 28x28 for MNIST and Fashion-MNIST, and 32x32 for CIFAR-10 and CIFAR-100. The number of input channels was configured accordingly (1 for grayscale, 3 for color).

Data loading was managed with PyTorch’s `DataLoader` class [38]. A consistent batch size of 128 was used for training and evaluation across nearly all experiments, and the FF algorithm constituted the sole exception, for which a batch size of 100 was used to align with its reference implementation.

3.4 Implementation of Alternative Algorithms

Each alternative algorithm was implemented to faithfully represent its original description, with a focus on the specific architectural and procedural details required for the experiments in this paper.

3.4.1 Forward-Forward (FF) Algorithm

The author’s implementation of the FF algorithm, conceptually detailed in Section 2.3.1, was configured for its native MLP architectures.

- **Architectural Details:**

- **For Fashion-MNIST:** A 4×2000 ReLU MLP was used.
- **For MNIST:** Both a 3×1000 and a 4×2000 ReLU MLP were implemented.

In all cases, layer normalization was applied as length normalization ($\mathbf{x}_l / (\|\mathbf{x}_l\|_2 + \epsilon)$) after each hidden layer’s ReLU activation.

- **Mechanism Implementation:**

- Positive and negative data were created by embedding one-hot-encoded labels into the first 10 pixels of the input images.
- The goodness function was implemented as the sum of squared activations.
- The local logistic loss used a dynamic threshold equal to the number of neurons in the layer.
- Both FF layers and a separate downstream classifier were trained using a shared optimizer (AdamW or SGD), with hyperparameters determined by Optuna.

- **Inference Implementation:** Final predictions were determined by adding goodness scores across all hidden layers for each class label and selecting the class with the highest total score.

3.4.2 Cascaded-Forward (CaFo) Algorithm

The CaFo algorithm, whose block-wise training paradigm is described in Section 2.3.2, was implemented for all datasets using its native CNN architecture.

- **Architectural Details:** A CNN with three blocks was implemented, where each block consisted of a 3×3 Convolution, a ReLU activation, a 2×2 Max Pooling layer, and Batch Normalization. The channel depths were set to 32, 128, and 512 for the respective blocks.
- **Predictor Implementation:** A dedicated predictor, consisting of a flattening operation followed by a single Fully Connected (FC) layer, was attached to each of the three blocks.
- **Training Variants Implemented:** The experimental evaluation focused on two primary variants both using Cross-Entropy (CE) loss for predictor training:
 - **CaFo-Rand-CE:** The CNN blocks were initialized with Kaiming Uniform and remained frozen during training.
 - **CaFo-DFA-CE:** The CNN blocks were first pre-trained using DFA, after which their weights were frozen for the subsequent predictor training phase.

- **Predictor Training Protocol:** Each predictor was trained independently using the Adam optimizer. The training process for each predictor was governed by its own early stopping mechanism based on validation performance, with hyperparameters tuned via Optuna.
- **Inference Implementation:** The final class prediction was generated by summing the logits from all three trained predictors.

3.4.3 Mono-Forward (MF) Algorithm

The implementation of the MF algorithm was based on its native MLP architectures, as detailed conceptually in Section 2.3.3.

- **Architectural Details:**
 - **For MNIST & Fashion-MNIST:** A 2×1000 ReLU MLP.
 - **For CIFAR-10 & CIFAR-100:** A 3×2000 ReLU MLP.

Each hidden layer was augmented with a learnable projection matrix ($\mathbf{M}_i$), and both the layer weights ($\mathbf{W}_i$) and the projection matrices were initialized using the Kaiming uniform method.

- **Training Protocol:** Training was conducted in a layer-wise fashion. For each layer, its weights and projection matrix were trained jointly using the Adam optimizer to minimize a local Cross-Entropy loss. This local loss was derived from the softmax of the goodness scores, which were calculated as described in Equation 2.7. Early stopping based on local validation loss determined the duration of training for each layer.
- **Inference Implementation:** The final prediction was made using the BP-style method, where only the goodness scores from the final hidden layer were used to generate the classification output.

3.5 Fair Backpropagation Baseline Setup

For each alternative algorithm experiment, a dedicated BP baseline was meticulously constructed.

- **Architectural Replication:** For every native architecture detailed in Section 3.4, an identical network structure was created for the BP baseline. This replication included the same number and type of layers, neuron/channel counts, ReLU activation functions, and normalization layers, such as BatchNorm retention in the CaFo CNN baseline.
- **Consistent Initialization:** The weights of all layers were initialized using the same method as their counterparts in the implementation of the alternative algorithm, primarily the Kaiming uniform method.
- **Removal of Algorithm-Specific Components:** All components unique to the alternative algorithms were removed. These included the label embedding mechanism of FF, the intermediate predictors of CaFo, the projection matrices $\mathbf{M}_i$ and local losses of MF, and specialized normalizations such as FF’s length normalization.
- **Standard BP Training:** The baselines were trained end-to-end using standard BP [42]. A final fully connected layer followed by Softmax activation was used for classification, with a global Cross-Entropy loss function being minimized. The AdamW optimizer [33] was used for training. Crucially, the BP baseline training also employed the same early

stopping mechanism as the alternative algorithms, monitoring validation accuracy and saving the checkpoint with the best performance to ensure a fair comparison based on the optimal performance achieved.

This configuration ensures that comparisons effectively isolate the influence of the training algorithm by accounting for the optimal training duration determined through early stopping.

3.6 Hyperparameter Optimization Strategy

To facilitate a fair comparison and allow each algorithm to perform effectively, a systematic hyperparameter optimization strategy was employed for all evaluated algorithms, including FF, CaFo, MF, and their corresponding BP baselines. This tuning was executed using the Optuna framework [2].

The general approach involved defining a search space for key hyperparameters relevant to each algorithm and utilizing Optuna’s Tree-structured Parzen Estimator (TPE) sampler [7] to explore this space over a specified number of trials, typically between 30 and 50. The objective of each trial was to maximize the model’s accuracy on the separate validation set. The training process within each Optuna trial was subject to the same early stopping protocol used in the final experimental runs. The optimal hyperparameters identified for each configuration were then used for the final training runs reported in Chapter 4. This consistent tuning methodology ensures a more robust comparison by mitigating the influence of suboptimal default parameters. For efficiency, detailed resource monitoring was generally disabled during the Optuna trials.

3.6.1 Alternative Algorithm Tuning

Dedicated Optuna objective functions were developed to tune each alternative algorithm. Key tuned parameters included:

- **Forward-Forward (FF):** Learning rates and weight decay values for both FF layer updates and separate downstream classifier updates.
- **Cascaded-Forward (CaFo):** Predictor learning rate, weight decay, and the number of training epochs per predictor. For the **CaFo-DFA** variant, the block learning rate, weight decay, and the number of block training epochs were also tuned.
- **Mono-Forward (MF):** The primary learning rate for updating both layer weights and projection matrices, and possibly the number of training epochs applied per layer.

For these algorithms, Optuna’s pruner was generally disabled, as their layer-wise or block-wise training dynamics are not consistently amenable to standard pruning mechanisms. Appendix A documents the best hyperparameters found for each configuration.

3.6.2 Backpropagation Baseline Tuning

The Optuna framework was also systematically applied to tune the hyperparameters for every BP baseline.

- **Optimizer:** The AdamW optimizer [33] was used.
- **Search Space:** The learning rate and weight decay were tuned over log-uniform distributions.
- **Objective:** The objective was to maximize validation set accuracy, with the training process within each trial also incorporating early stopping.

- **Procedure:** Optuna was executed for a set number of trials for each BP baseline configuration. A Median pruner was occasionally employed to discard unpromising trials early. The hyperparameters with the best performance were used for the final BP baseline runs, with results documented in Appendix A.

This process ensures that the BP baselines operate near their optimal performance level on the specific replicated architectures, thereby allowing a fair assessment of the alternative algorithms.

3.6.3 Rationale for Universal Tuning

The decision was made to apply systematic hyperparameter optimization to **all** training algorithms to ensure the most rigorous and fair comparison possible. Although alternative algorithms are sometimes presented with specific default parameters, these may not be optimal for different datasets, architectures, or the specific training procedures employed in this work. This universal tuning aimed to:

- **Ensure Fairness:** Prevent a situation where an algorithm underperforms merely due to suboptimal hyperparameters.
- **Assess Potential:** Evaluate each algorithm closer to its performance limit as determined by validation accuracy in conjunction with early stopping.
- **Isolate Algorithmic Effects:** More effectively isolate the impact of the core training mechanism from the confounding factor of hyperparameter choice.

This universal tuning approach was deemed essential to draw robust conclusions about the relative trade-offs of these novel training paradigms. The resulting hyperparameters are documented in Appendix A.

3.6.4 A Note on "Effective Epochs" and Training Duration

A direct consequence of applying early stopping in both end-to-end and layer-wise training paradigms is that the total training effort, measured in epochs, is not directly comparable. For this reason, the term "Effective Epochs" is introduced to quantify this effort, with the following distinct interpretations:

- **For Backpropagation (BP) and Forward-Forward (FF):** Effective Epochs signifies the total number of *global training epochs* completed for the entire network before the early stopping criterion was met.
- **For Mono-Forward (MF):** Effective Epochs represent the *sum of the actual number of epochs for which each individual layer was trained*, subject to early stopping based on its own local validation loss.
- **For Cascaded-Forward (CaFo):** Effective Epochs are the *sum of the actual epochs for which each predictor was trained* (in the CaFo-Rand-CE variant), or the sum of epochs for block training plus the sum of epochs for predictor training (in the CaFo-DFA-CE variant).

This distinction is crucial for an accurate interpretation of the convergence behavior and training times presented in Chapter 4.

3.7 Metrics for Evaluation

A comprehensive set of metrics was used to evaluate both performance of the model and computational efficiency:

- **Performance Metrics:**
 - **Classification Accuracy:** The percentage of correct classifications on the test set, calculated using the model checkpoint that achieved the **best validation accuracy** during training, determined by the early stopping procedure.
 - **Convergence Rate:** Plotted as accuracy or loss against wall-clock time to visualize the training process until its termination.
 - **Loss Curves:** Plots of training and validation loss versus epochs.
- **Efficiency Metrics:**
 - **Energy Consumption:** The total GPU energy consumed during the main training loop, measured from the start of training **up to the point of termination**. This was measured in Watt hours (Wh) using the NVML API by integrating the power draw over time.
 - **Training Time:** The wall-clock time for the main training loop **until termination**, reported in seconds (s).
 - **Peak Memory Usage:** The maximum allocated GPU memory during the effective training duration, measured in Mebibytes (MiB) using NVML.
 - **Floating Point Operations (FLOPs):** Estimated using PyTorch’s profiler:
 - * **Estimated Forward Pass GFLOPs (F_{fwd}):** The GFLOPs required for a single forward pass on one sample, used to quantify inference complexity.
 - * **Estimated BP Update Cycle GFLOPs (F_{BP_update}):** Calculated *exclusively* for BP baselines to estimate the cost of a full update cycle, a using the heuristic detailed in Section 2.2 (Equation 1).
 - **Estimated Carbon Emissions (gCO₂e):** The estimated CO₂e generated during the effective training duration. These were calculated using the CodeCarbon library in **offline mode**, with a specified grid carbon intensity factor corresponding to the hardware’s location (Poland: "POL").

These metrics provide a multifaceted view for analyzing the energy performance trade-offs that result from training procedures optimized via early stopping.

3.8 Experimental Infrastructure and Tools

All experiments were performed on the **Athena cluster**, a high-performance computing (HPC) resource provided by ACK Cyfronet AGH.

- **Hardware:** Computations were conducted on the `plgrid-gpu-a100` partition. Each experimental run was allocated a **single NVIDIA A100-SXM4 40GB GPU**. The cluster nodes are equipped with AMD EPYC 7742 CPUs.
- **Job Scheduling:** Jobs were managed via the **Slurm Workload Manager** and submitted to the `plgrid-gpu-a100` partition under the `plgoncotherapy-gpu-a100` grant.
- **Configuration Management:** A structured system of **YAML files** was used, where configurations for each experiment were inherited from a base file defining common defaults.

- **Deep Learning Framework and Environment:** The software environment was built upon loaded modules for **Python/3.10.4** and **CUDA/12.4.0**. The key libraries installed within a virtual environment included **PyTorch v2.4.0**, **Optuna v4.2.1**, **CodeCarbon v3.0.1**, and **pynvml v12.0.0**.
- **Energy/Resource Monitoring:** NVML was utilized to monitor GPU energy consumption, power consumption, and maximum memory usage.
- **FLOPs Profiling:** Forward pass FLOPs were estimated with **torch.profiler**.
- **Carbon Footprint Estimation:** The CodeCarbon library was used in its offline mode with the ISO code "POL" to estimate CO2e emissions.
- **Experiment Tracking:** Weights & Biases (**wandb**) served as the primary tool to log metrics and track results.
- **Version Control:** Git and GitHub were used for version control. The code repository is detailed in Appendix C.

Detailed environment specifications are provided in Appendix B.

4 Experimental Results

This chapter presents the empirical findings from the comparative experiments conducted according to the fair benchmarking framework and methodology detailed in Chapter 3. The results focus on evaluating the performance and energy efficiency of the alternative algorithms against their respective fair backpropagation (BP) baselines across the MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100 datasets.

Although performance is primarily assessed through the accuracy of classification and convergence behavior, efficiency evaluation utilizes a multifaceted approach. To provide a robust assessment of practical computational and energy efficiency, the analysis prioritizes directly measured hardware metrics. These primary indicators include total energy consumption, measured in Watt-hours (Wh) using the NVML API, the wall-clock training time until termination by early stopping, and the peak GPU memory usage, also monitored via NVML. These measurements reflect the utilization of real-world resources on the specified NVIDIA A100 hardware platform. To supplement these hardware-specific metrics, estimated GFLOPs for a single forward pass (F_{fwd}), obtained using the PyTorch Profiler, are reported to provide a hardware-agnostic measure of arithmetic complexity. For BP baselines, the estimated GFLOPs per update cycle ($F_{BP_update} \approx 3 \times F_{fwd}$) are included to illustrate the theoretical computational difference that arises from the backward pass. Furthermore, an estimated CO2e is reported, calculated to provide an environmental impact perspective based on the measured energy consumption within the specified compute region (Poland, ISO code: POL). The subsequent sections detail the results for each algorithm and its corresponding baseline, followed by comparative analyses focusing on these key performance and efficiency indicators.

4.1 Forward-Forward (FF) vs. Backpropagation (BP) Comparison

This section presents a foundational analysis comparing Geoffrey Hinton’s FF algorithm against a fair BP baseline, focusing on the MLP architectures for which FF was primarily designed. The initial performance results reveal a significant paradox: while FF can achieve impressive accuracy, thus validating its core concept, it does so at a drastically higher convergence cost.

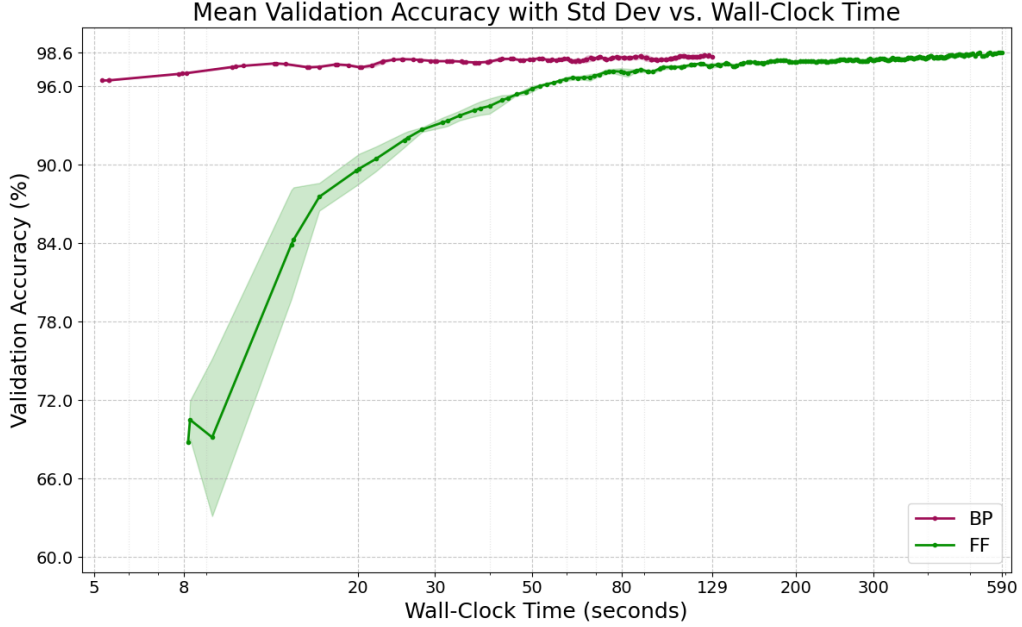
Table 1: Performance Comparison: FF vs. BP on MLP Architectures (Mean $\pm$ Std Dev over 3 runs).

Dataset	Architecture	Algorithm	Test Accuracy (%)	Effective Epochs
F-MNIST	4×2000 MLP	FF-AdamW	89.63 $\pm$ 0.12	90.33 $\pm$ 16.86
		BP Baseline	88.88 $\pm$ 0.61	15.67 $\pm$ 0.58
MNIST	3×1000 MLP	FF-AdamW	98.33 $\pm$ 0.03	77.00 $\pm$ 0.00
		FF-SGD	98.23 $\pm$ 0.05	99.33 $\pm$ 1.15
		BP Baseline	98.09 $\pm$ 0.06	35.00 $\pm$ 0.00
MNIST	4×2000 MLP	FF-AdamW	98.42 $\pm$ 0.14	87.00 $\pm$ 11.53
		BP Baseline	98.43 $\pm$ 0.03	49.00 $\pm$ 0.00

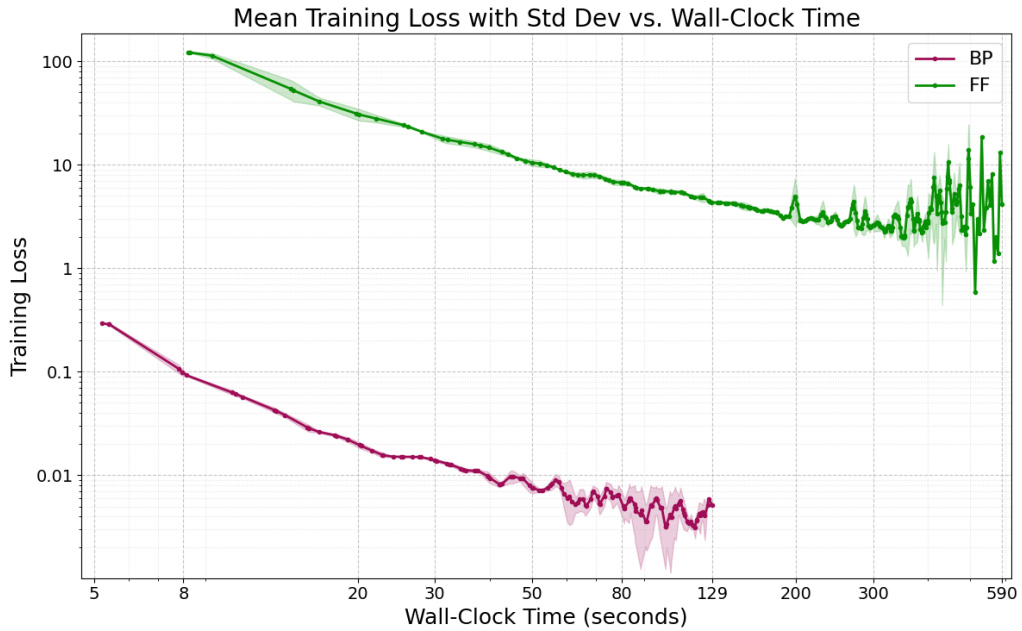
Effective Epochs: Average number of global training epochs completed before early stopping. Best results in each comparison group are highlighted in **bold**. For accuracy, higher is better; for epochs, lower is better.

As shown in Table 1, the FF algorithm successfully trains MLP architectures to a final test accuracy that is competitive with, and in some cases slightly superior to, a fairly tuned BP baseline on both the MNIST and Fashion-MNIST datasets. This result is significant, as it validates the core premise of Hinton’s work: effective learning is indeed possible without propagating error gradients backwards.

However, the Effective Epochs metric in Table 1 reveals the steep cost of this accuracy. FF consistently required a dramatically higher number of training epochs to reach its optimal validation checkpoint, often more than double that of BP, indicating a far less efficient learning process. The practical implications of this inefficiency are best understood by analyzing the convergence behavior with respect to the actual wall-clock time.



(a)



(b)

Figure 1: Convergence Dynamics for FF vs. BP on MNIST 4×2000 MLP.

Figure 1 provides a clear representation of this inefficiency. The validation accuracy plot (Figure 1a) shows that the BP baseline achieves near-optimal performance rapidly, in well under 100 seconds, with a stable and consistent learning trajectory. In stark contrast, FF embarks on a slow, gradual, and more volatile learning path, which requires more than 400 seconds to reach a similar accuracy level. The training loss plot (Figure 1b) reveals the underlying

algorithmic cause for this disparity: BP’s loss decreases smoothly and efficiently, whereas FF’s loss decreases slowly and erratically from a much higher starting point, ultimately plateauing at a level orders of magnitude above that of BP. This indicates that FF’s collection of local "goodness" objectives serves as a far weaker and less direct proxy for the global classification task, resulting in a fundamentally less efficient optimization landscape that necessitates a protracted training process with severe consequences for overall resource consumption.

The slow convergence of the FF algorithm directly translates into a significant increase in computational cost across all measured efficiency metrics. As detailed in Table 2, the training time required to reach the best model was substantially longer (a 4x to 13x increase) and the total energy consumption was significantly higher (a 3x to 10x increase) for FF compared to BP across all configurations. This led to a correspondingly larger estimated CO2e footprint.

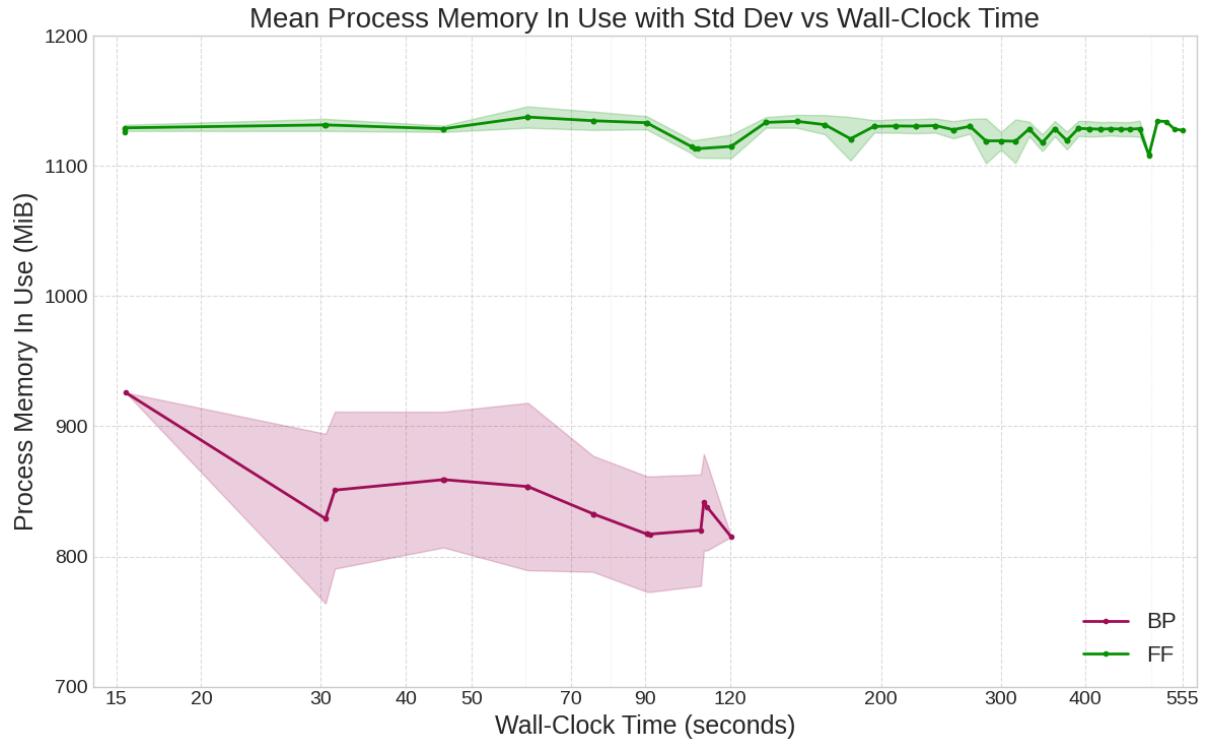
Table 2: Efficiency Comparison: FF vs. BP on MLP Architectures (Mean $\pm$ Std Dev over 3 runs).

Dataset	Architecture	Algorithm	Train Time (s)	Energy (Wh)	Est. CO2e (g)	Peak Mem (MiB)	Total GFLOPs
F-MNIST	4×2000 MLP	FF-AdamW	574.60 $\pm$ 134.49	14.28 $\pm$ 2.66	23.10 $\pm$ 8.52	1190	0.027
		BP Baseline	43.09 $\pm$ 0.80	1.48 $\pm$ 0.01	1.67 $\pm$ 0.01	1168	0.082
MNIST	3×1000 MLP	FF-AdamW	370.72 $\pm$ 8.63	6.68 $\pm$ 0.30	10.44 $\pm$ 0.82	954	0.006
		FF-SGD	444.15 $\pm$ 5.07	7.89 $\pm$ 0.16	12.26 $\pm$ 0.73	956	0.006
		BP Baseline	83.85 $\pm$ 1.12	1.52 $\pm$ 0.01	2.55 $\pm$ 0.29	946	0.017
MNIST	4×2000 MLP	FF-AdamW	509.59 $\pm$ 73.20	14.26 $\pm$ 1.74	17.50 $\pm$ 2.74	1190	0.027
		BP Baseline	125.80 $\pm$ 3.13	4.23 $\pm$ 0.11	5.25 $\pm$ 0.07	1168	0.082

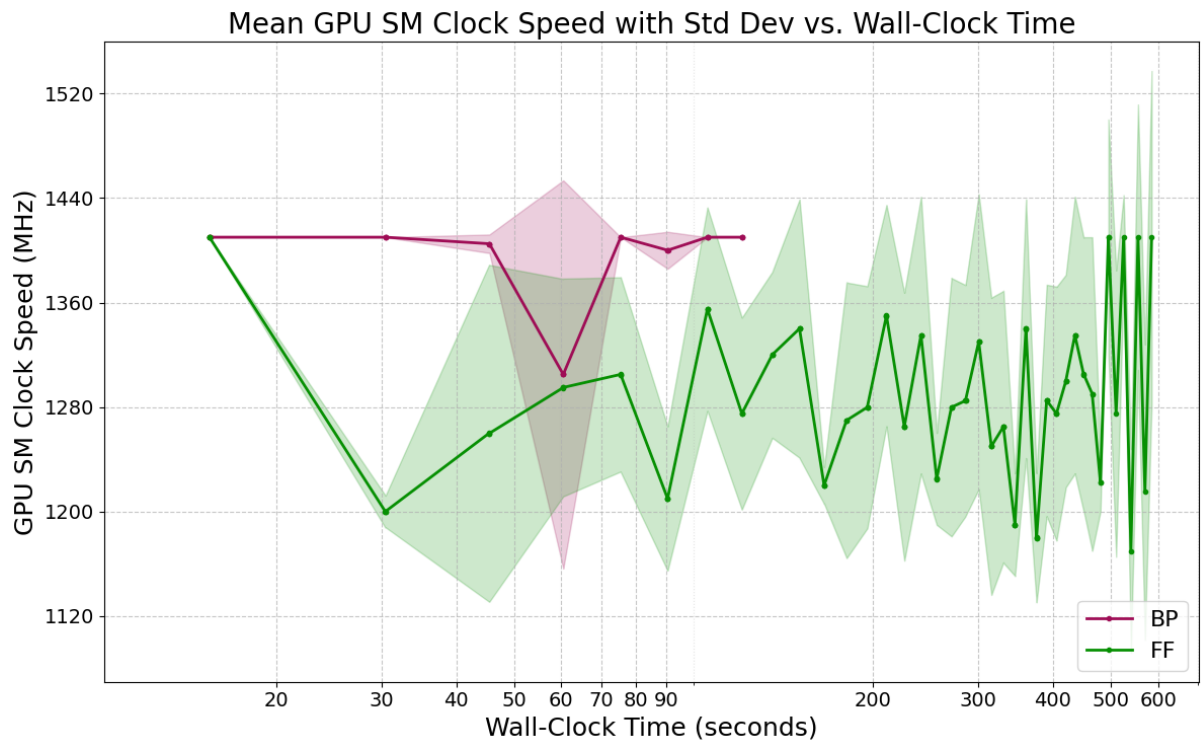
Total GFLOPs: Estimated GigaFLOPs per sample. For FF, this is the cost of a single forward pass (F_{fwd}). For BP, it is the estimated cost of a full update cycle (F_{BP_update}). Best efficiency results in each comparison group are highlighted in **bold** (lower is better).

A definitive explanation for this inefficiency emerges from an analysis of the hardware-level monitoring plots in Figure 2. The memory plot (Figure 2a) reveals a critical and counter-intuitive finding: contrary to the theoretical benefit of avoiding activation storage, **the FF algorithm consistently used slightly more GPU peak memory than BP in these experiments**. This observation empirically refutes a key presumed advantage, suggesting that practical implementation overheads, such as managing separate local optimizers, handling positive and negative data streams, or other specific PyTorch implementation details, impose a memory cost that outweighs the theoretical savings for these MLPs. More revealing is the GPU SM clock speed plot (Figure 2b), which provides a clear indicator of FF’s inefficiency. While BP maintains a high, stable GPU clock speed, indicating that it effectively saturates the hardware with large, parallelizable matrix multiplications, FF’s clock speed is erratic, "spiky," and operates at a lower average. This pattern signifies a computational bottleneck in which the GPU is frequently stalled, likely due to a high volume of small, sequential operations that lead to poor hardware utilization. Consequently, **FF’s slowness is not due to its computations being inherently more complex, but to its failure to efficiently utilize modern, highly parallel hardware**.

In summary, the FF algorithm serves as a vital foundational exploration, confirming its conceptual validity by training MLP architectures to an accuracy competitive with backpropagation. However, this success comes at a severe and disproportionate efficiency cost. The combination of a less effective optimization process and inefficient hardware utilization results in drastically longer training times, higher energy consumption, and a larger estimated CO2e footprint, while theoretical memory savings were not realized in this practical context. These findings quantify FF’s shortcomings and motivate the investigation of more structured approaches like CaFo, which aim to resolve these inefficiencies by integrating supervisory signals more directly.



(a)



(b)

Figure 2: GPU Resource Utilization Profiles for FF vs. BP on MNIST 4×2000 MLP.

4.2 Cascaded-Forward (CaFo) vs. Backpropagation (BP) Comparison

This section presents a holistic analysis of the CaFo algorithm on its native 3-block CNN architecture, moving beyond foundational concepts to evaluate its practical viability as a structured BP-free alternative. The investigation covers two primary variants, CaFo-Rand-CE (using randomly initialized, fixed blocks) and CaFo-DFA-CE (using blocks pre-trained with DFA), against a fair BP baseline. By integrating dynamic convergence behavior with final, static performance, and efficiency metrics, a complete picture of the algorithm’s inherent trade-offs is constructed. To understand CaFo’s practical efficiency, it is crucial to first analyze the distinct convergence behaviors of its different variants.

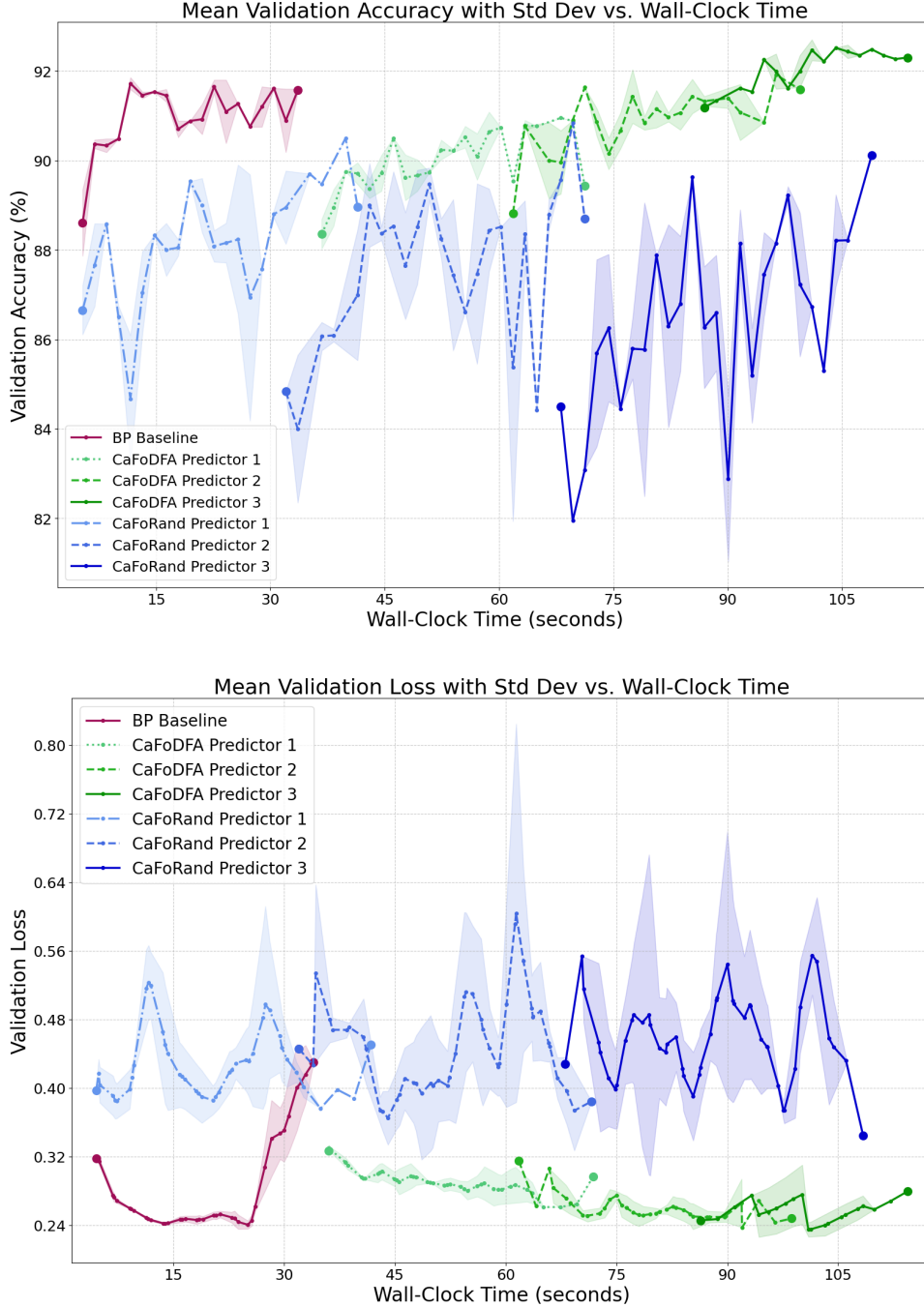


Figure 3: Convergence Dynamics for CaFo Variants and BP on Fashion-MNIST 3-block CNN.

Table 3: Performance Comparison: CaFo Variants vs. BP on Native 3-block CNN (Mean $\pm$ Std Dev over 3 runs).

Dataset	Architecture	Algorithm	Test Accuracy (%)	Total Effective Epochs
MNIST	3-block CNN	CaFo-Rand-CE	98.62 $\pm$ 0.02	30.00 $\pm$ 0.00
		CaFo-DFA-CE	99.02 $\pm$ 0.05	37.00 $\pm$ 1.00
		BP Baseline	98.94 $\pm$ 0.15	15.33 $\pm$ 0.58
F-MNIST	3-block CNN	CaFo-Rand-CE	90.36 $\pm$ 0.45	44.33 $\pm$ 2.08
		CaFo-DFA-CE	91.72 $\pm$ 0.12	47.33 $\pm$ 3.51
		BP Baseline	89.25 $\pm$ 0.34	13.33 $\pm$ 0.58
CIFAR-10	3-block CNN	CaFo-Rand-CE	66.09 $\pm$ 0.32	72.67 $\pm$ 15.01
		CaFo-DFA-CE	77.60 $\pm$ 0.25	295.00 $\pm$ 6.56
		BP Baseline	79.32 $\pm$ 3.99	71.67 $\pm$ 3.79
CIFAR-100	3-block CNN	CaFo-Rand-CE	41.19 $\pm$ 0.29	136.33 $\pm$ 9.81
		CaFo-DFA-CE	48.20 $\pm$ 0.22	261.33 $\pm$ 11.37
		BP Baseline	52.63 $\pm$ 5.20	37.33 $\pm$ 2.52

Total Effective Epochs: Sum of actual epochs each component was trained for. Best results in each comparison group are in **bold**. For accuracy, higher is better; for epochs, lower is better.

The convergence plots in Figure 3 offer a granular view of the learning process. The BP baseline exemplifies an ideal learning trajectory, exhibiting rapid, smooth, and stable convergence to a high-accuracy plateau, showcasing the efficacy of end-to-end optimization guided by a global error signal. In contrast, the CaFo-DFA-CE variant demonstrates a clear two-phase process: an initial period of no predictor activity represents the upfront computational cost of pre-training the feature extraction blocks via DFA, after which the individual predictors begin training sequentially and learn quickly, with the predictor leveraging the deepest features ultimately achieving the highest accuracy. However, the CaFo-Rand-CE variant exhibits a highly volatile and inefficient learning process, with accuracy fluctuating wildly, indicating that the untrained random features are insufficient for stable credit assignment. The loss curves corroborate these findings, with BP’s loss decreasing rapidly to a low, stable minimum, while the loss for CaFo-Rand-CE remains high and erratic. The crucial insight from this dynamic analysis is the trade-off between feature quality and computational cost: the superior performance of CaFo-DFA comes at the price of a significant upfront time and energy investment in block training. The failure of the CaFo-Rand variant underscores that effective BP-free learning cannot be achieved by simply attaching local classifiers to random projections; the quality of intermediate representations is paramount.

The dynamic behaviors are quantified in the final performance and efficiency metrics, which provide a static comparison of the final outcomes and allow a holistic assessment.

The test accuracy results in Table 3 confirm this performance hierarchy, with backpropagation setting a strong benchmark. CaFo-DFA-CE emerges as the most competitive alternative, significantly closing the accuracy gap on complex datasets like CIFAR-10 and even surpassing BP on less complex datasets like Fashion-MNIST. In contrast, CaFo-Rand-CE consistently lags in performance, especially on CIFAR-10 and CIFAR-100, where it suffers a substantial accuracy drop of over 13 and 11 percentage points, respectively. This finding is directly in line with the work of [52], who also demonstrated the superiority of using trained blocks.

However, the central trade-off of the CaFo algorithm becomes evident in the efficiency metrics detailed in Table 4.

Table 4: Efficiency Comparison: CaFo Variants vs. BP on Native 3-block CNN (Mean $\pm$ Std Dev over 3 runs).

Dataset	Algorithm	Train Time (s)	Energy (Wh)	Est. CO2e (g)	Peak Mem (MiB)	Total GFLOPs
MNIST	CaFo-Rand-CE	69.20 $\pm$ 0.69	1.20 $\pm$ 0.02	2.06 $\pm$ 0.25	1008	0.07271
	CaFo-DFA-CE	84.52 $\pm$ 2.24	1.64 $\pm$ 0.05	2.74 $\pm$ 0.49	1092	0.07271
	BP Baseline	37.61 $\pm$ 0.18	0.90 $\pm$ 0.02	1.33 $\pm$ 0.04	1080	0.21840
F-MNIST	CaFo-Rand-CE	101.09 $\pm$ 3.55	1.76 $\pm$ 0.07	2.80 $\pm$ 0.05	1008	0.07271
	CaFo-DFA-CE	105.37 $\pm$ 7.40	2.48 $\pm$ 0.11	3.51 $\pm$ 0.27	1092	0.07271
	BP Baseline	33.18 $\pm$ 0.94	0.81 $\pm$ 0.04	1.13 $\pm$ 0.06	1080	0.21840
CIFAR-10	CaFo-Rand-CE	332.79 $\pm$ 61.94	5.50 $\pm$ 1.00	10.22 $\pm$ 1.10	1014	0.09614
	CaFo-DFA-CE	1314.38 $\pm$ 37.66	27.37 $\pm$ 0.44	45.71 $\pm$ 2.14	1128	0.09614
	BP Baseline	339.51 $\pm$ 18.66	6.81 $\pm$ 0.46	10.93 $\pm$ 1.28	1114	0.28893
CIFAR-100	CaFo-Rand-CE	631.60 $\pm$ 38.83	10.56 $\pm$ 0.77	18.62 $\pm$ 0.33	1060	0.09614
	CaFo-DFA-CE	1161.79 $\pm$ 60.05	24.09 $\pm$ 1.32	39.23 $\pm$ 4.88	1144	0.09614
	BP Baseline	180.57 $\pm$ 11.23	3.56 $\pm$ 0.17	5.63 $\pm$ 0.36	1116	0.29334

Total GFLOPs: Estimated GigaFLOPs per sample (F_{fwd} for CaFo, F_{BP_update} for BP). Best efficiency results in each group are in **bold** (lower is better).

The results reveal the substantial cost of CaFo-DFA-CE’s accuracy; on CIFAR-10, for instance, it required approximately four times more training time and energy and generated four times more estimated CO2e than the BP baseline, all while achieving slightly lower accuracy. This highlights the critical weakness of the CaFo-DFA algorithm: its method for BP-free feature learning, while effective, is computationally expensive. The CaFo-Rand-CE variant, on the other hand, illuminates a potential efficiency niche, proving approximately 19% more energy efficient than BP on CIFAR-10, but this came at the cost of a 13 percentage point drop in accuracy.

A particularly nuanced finding emerges from the analysis of peak memory consumption. The CaFo-Rand-CE variant consistently validates the theoretical benefit of BP-free methods, using 5 to 9% less peak memory than the BP baseline by avoiding the storage of activations for a deep backward pass. However, in a counterintuitive result, the CaFo-DFA-CE variant consistently consumed more peak memory than BP. This empirically demonstrates that the memory overhead of the DFA training mechanism itself, such as storing fixed random feedback matrices, can exceed the savings gained from eliminating the global backward pass, thus refuting any generalized assumption that all BP-free methods are inherently more memory-efficient.

A comparison with the original CaFo paper reveals both consistencies and notable differences attributable to the rigorous methodology employed in this study. The author’s implementation of CaFo-DFA-CE achieved notably better accuracy on CIFAR-10 (22.40% error vs. 30.52%) and CIFAR-100 (51.80% error vs. 57.87%) while also training significantly faster. These improvements are likely attributable to the systematic use of Optuna for hyperparameter tuning and the consistent application of early stopping for each predictor based on its validation loss, which allows each component to train to its optimal point and avoids the fixed, and potentially excessive, epoch counts used in the original work.

In the broader landscape of BP-free learning, **the CaFo algorithm represents a significant conceptual advance over simpler ideas such as FF**. Its structured block-wise approach allows it to achieve respectable accuracy on complex CNN architectures. Despite these architectural innovations, the empirical results show that CaFo does not present a universally superior alternative to backpropagation. Instead, it presents the practitioner with a stark choice: backpropagation offers the best overall balance of high accuracy and outstanding time and energy efficiency; the CaFo-DFA-CE variant can achieve near-BP accuracy but at a prohibitive computational cost, making it impractical for most energy-constrained applications; and finally, the CaFo-Rand-CE variant offers modest memory and potential energy benefits but

with a significant performance penalty on complex tasks. The challenges highlighted by CaFo’s trade-off profile provide a strong motivation to investigate other advanced algorithms like MF, which aims to achieve BP-competitive accuracy with a more efficient local learning mechanism, thus setting the stage for the next phase of this investigation.

4.3 Mono-Forward (MF) vs. Backpropagation (BP) Comparison

This section presents a comprehensive empirical evaluation of the MF algorithm, the final and most advanced BP-free method investigated in this paper, focusing on its performance and efficiency in its native MLP architectures. The results robustly demonstrate that for the MLP architectures under investigation, **MF is not merely a theoretical alternative but a practical, high performance and computationally efficient training algorithm**. It consistently achieves superior accuracy while demanding significantly fewer computational resources, establishing it as a compelling alternative to conventional backpropagation for this class of networks.

The empirical evidence indicates that MF’s local, greedy learning approach consistently matches or slightly surpasses the classification accuracy of a fairly tuned BP baseline on identical MLP structures. This performance advantage is most pronounced on the more complex CIFAR-10 task, where MF achieves a notable 1.21 percentage point lead, as detailed in Table 5.

Table 5: Performance Comparison: MF vs. BP on Native MLP Architectures (Mean $\pm$ Std Dev over 3 runs).

Dataset	Architecture	Algorithm	Test Accuracy (%)	Effective Epochs
MNIST	2×1000 MLP	MF	98.14 ± 0.02	15.33 ± 0.58
		BP	98.05 ± 0.10	16.00 ± 0.00
F-MNIST	2×1000 MLP	MF	89.72 ± 0.11	23.00 ± 0.00
		BP	89.21 ± 0.50	17.67 ± 1.15
CIFAR-10	3×2000 MLP	MF	62.34 ± 0.05	38.00 ± 0.00
		BP	61.13 ± 0.34	50.67 ± 2.08
CIFAR-100	3×2000 MLP	MF	30.31 ± 0.15	24.00 ± 0.00
		BP	29.94 ± 0.37	21.33 ± 2.31

Effective Epochs: Sum of layer-wise epochs for MF, global epochs for BP. Best results in each comparison group are in **bold**. For accuracy, higher is better; for epochs, lower is better.

The key explanation for this superior generalization lies in the dynamics of the validation loss, illustrated in Figure 4, which shows that the sequence of local optimizations performed by MF converges to a more favorable and lower final validation loss compared to the best loss achieved by the end-to-end global optimization of backpropagation. **This suggests that MF’s layer-wise cascade is a more effective optimization strategy for these MLP architectures, leading to better generalization on the test set.**

However, the most significant findings relate to MF’s profound computational and energy efficiency gains. The algorithm demonstrates a marked advantage in the time required to reach its optimal validation checkpoint, proving approximately 33.8% faster than its BP counterpart on the CIFAR-10 MLP. The combination of lower power consumption and shorter training duration results in substantial energy savings. On the CIFAR-10 task, MF consumed approximately 40.8% less energy and produced 39.2% less estimated CO₂e than BP did, highlighting its potential, as quantified in Table 6.

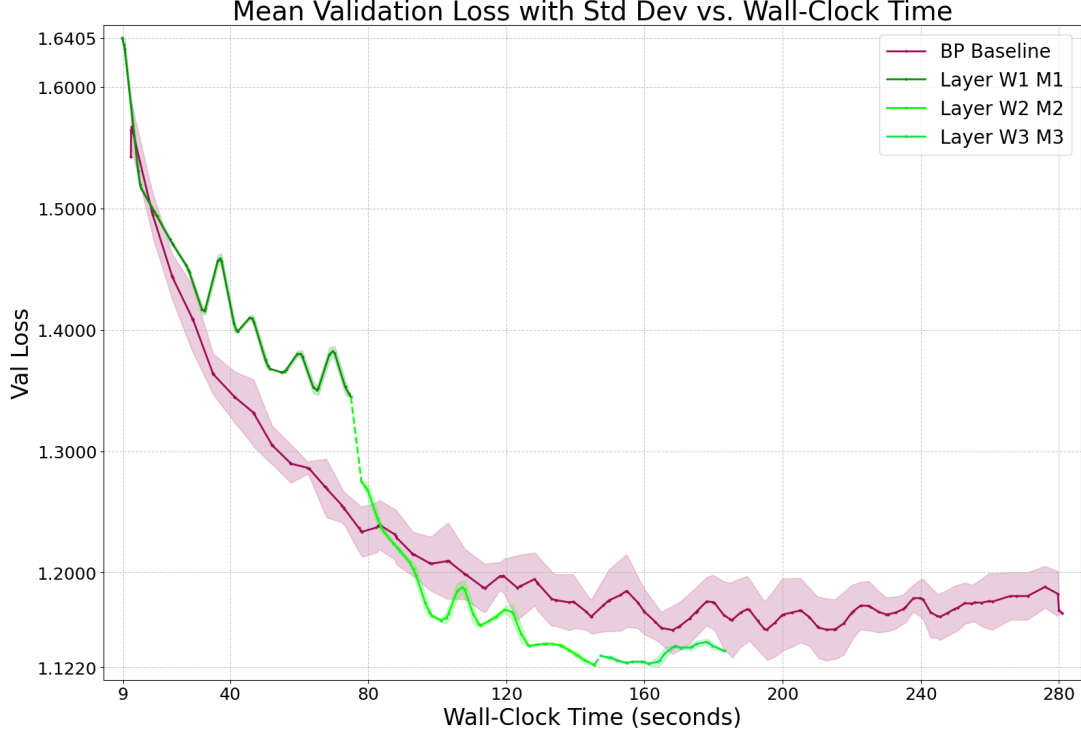


Figure 4: Mean Validation Loss Dynamics for MF vs. BP on CIFAR-10 3×2000 MLP.

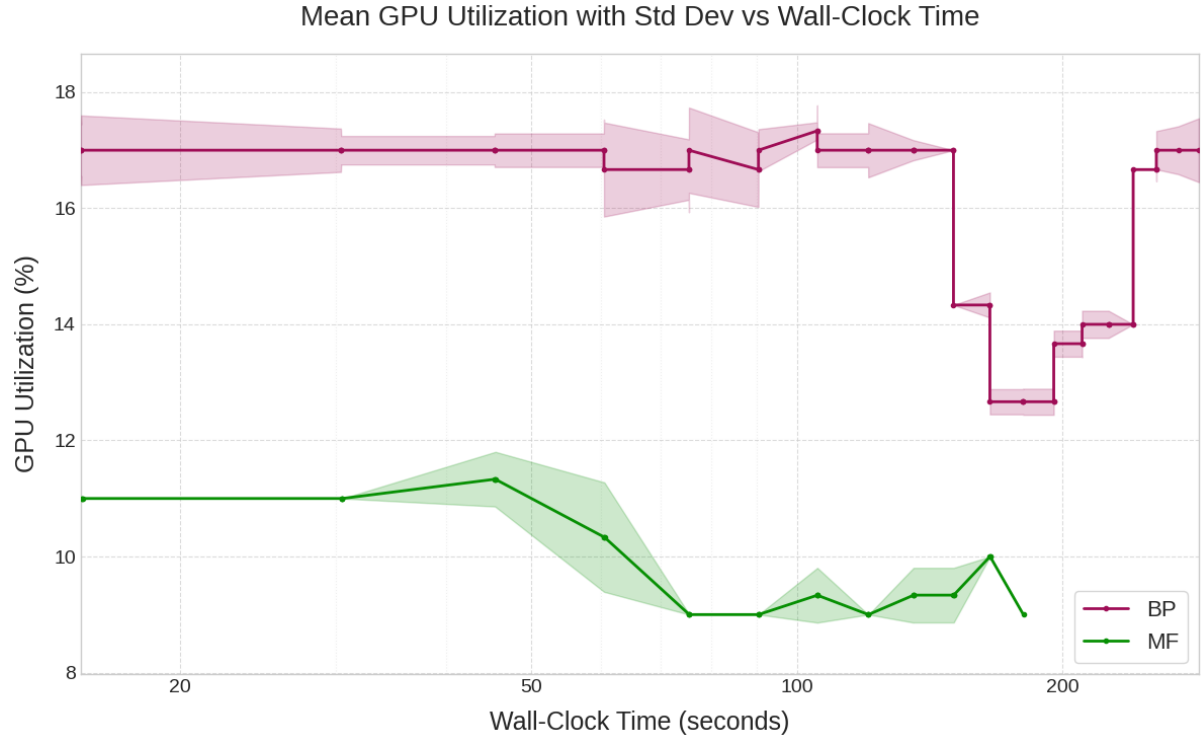
Table 6: Efficiency Comparison: MF vs. BP on Native MLP Architectures (Mean $\pm$ Std Dev over 3 runs).

Dataset	Architecture	Algorithm	Train Time (s)	Energy (Wh)	Est. CO2e (g)	Peak Mem (MiB)	Total GFLOPs
MNIST	2×1000 MLP	MF	35.03 $\pm$ 1.51	0.60 $\pm$ 0.03	1.15 $\pm$ 0.08	934	0.00359
		BP	39.84 $\pm$ 0.20	0.69 $\pm$ 0.00	1.30 $\pm$ 0.04	926	0.01077
F-MNIST	2×1000 MLP	MF	52.08 $\pm$ 0.43	0.86 $\pm$ 0.03	1.64 $\pm$ 0.34	934	0.00359
		BP	44.06 $\pm$ 4.45	0.79 $\pm$ 0.11	1.62 $\pm$ 0.57	926	0.01077
CIFAR-10	3×2000 MLP	MF	177.70 $\pm$ 1.62	3.17 $\pm$ 0.06	6.70 $\pm$ 0.71	1120	0.02833
		BP	268.45 $\pm$ 11.47	5.35 $\pm$ 0.22	11.03 $\pm$ 0.45	1184	0.08499
CIFAR-100	3×2000 MLP	MF	110.36 $\pm$ 1.33	2.02 $\pm$ 0.03	3.46 $\pm$ 0.35	1142	0.02869
		BP	111.78 $\pm$ 12.44	2.30 $\pm$ 0.26	3.29 $\pm$ 0.48	1192	0.08607

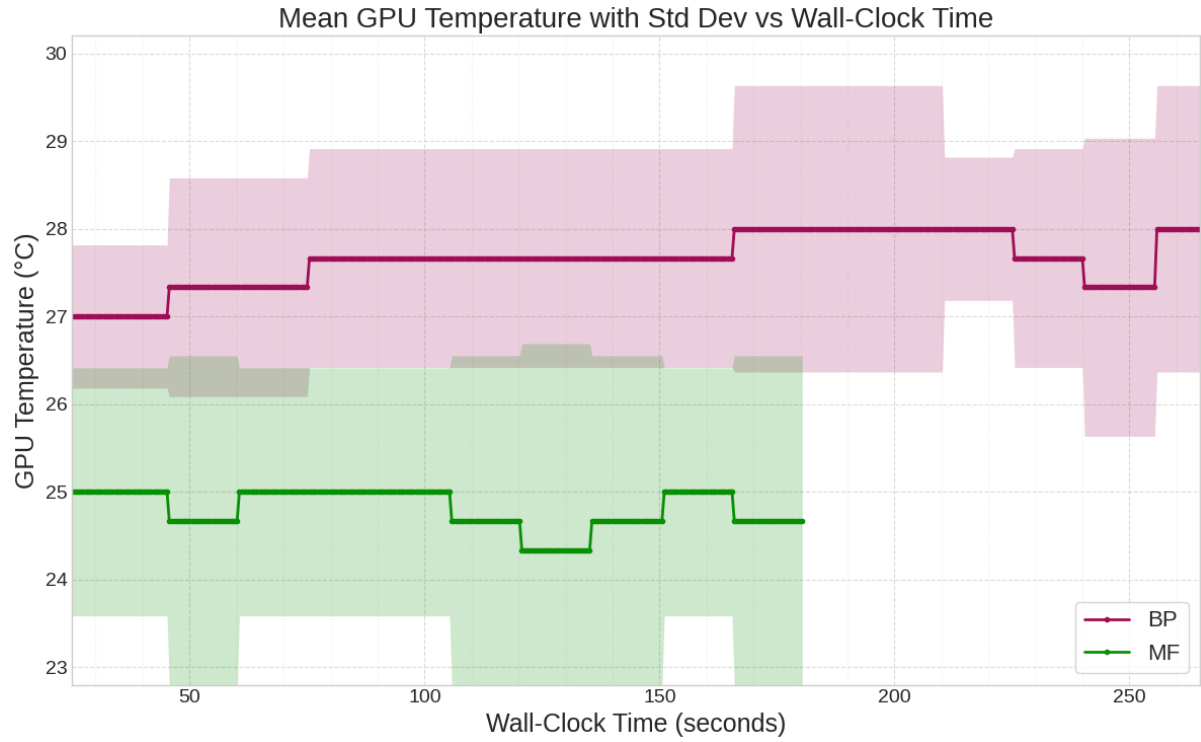
Total GFLOPs: Estimated per sample (F_{fwd} for MF, F_{BP_update} for BP). Best efficiency results in each group are in **bold** (lower is better).

This superior efficiency is a direct consequence of its computationally lighter design. Hardware-level monitoring reveals that MF operates at a significantly lower computational intensity (9 to 11% GPU utilization) compared to BP (16 to 17%), a direct result of eliminating the backward pass, as seen in Figure 5a. This reduced computational load leads to a reduced thermal profile, which is confirmed by the GPU temperature readings in Figure 5b.

Although the efficiency gains in time and energy are profound, the analysis of peak memory consumption presents a more nuanced picture. Contrary to the common expectation that BP-free methods yield large memory savings, empirical evidence shows that the MF’s advantage in this domain is modest. It realizes maximum memory reductions of approximately 4 to 5% on the larger MLPs, while incurring a slight increase on the smaller ones (Table 6).



(a)



(b)

Figure 5: Hardware-level Efficiency Indicators for MF vs. BP Training on CIFAR-10 3×2000 MLP.

This important finding highlights that the theoretical memory benefit of avoiding intermediate activation storage is partially offset by the practical memory overhead of MF’s unique components: the learnable projection matrix ($\mathbf{M}_i$) associated with each layer and the memory required for its corresponding optimizer states. This result qualifies generalized claims and underscores the necessity of empirical measurement to develop a complete understanding of an algorithm’s resource profile. Furthermore, the dynamic monitoring plots serve as powerful, independent signatures that validate MF’s unique training mechanism is operating as designed. The dynamic memory plot in Figure 6 provides a clear visual narrative, where the distinct step-like increase in MF’s memory usage aligns perfectly with the sequential initiation of each layer’s training.

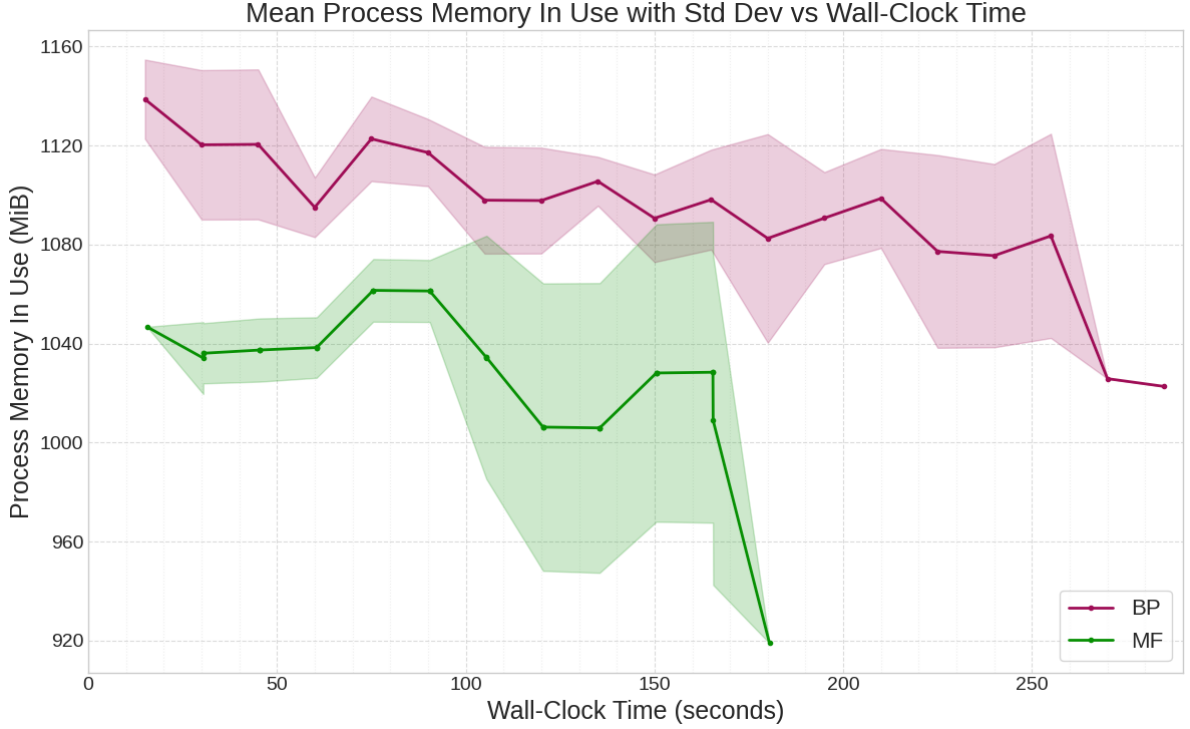


Figure 6: Mean Process Memory In Use for MF vs. BP on CIFAR-10 3×2000 MLP.

4.4 Cross Algorithm Comparative Synthesis

To synthesize the empirical findings detailed in the preceding sections, this analysis provides a consolidated overview of the performance and efficiency results across all evaluated algorithms. Table 7 quantifies the relative differences between each alternative training method and its corresponding fair backpropagation baseline, calculated from the mean results of three experimental runs. This comparative summary enables a direct assessment of the trade-offs inherent in each approach when implemented on its native architecture.

The results consolidated in Table 7 crystallize the main findings of this chapter. The Forward-Forward algorithm proved to be prohibitively inefficient, despite validating the foundational premise of BP-free learning by achieving an accuracy comparable to that of backpropagation on MLPs. Its training time and energy costs were consistently and substantially higher, without realizing any peak memory advantage in these experiments. The Cascaded-Forward algorithm, evaluated on its native CNNs, revealed a nuanced performance profile contingent on its implementation variant. The CaFo-Rand-CE variant demonstrated modest efficiency gains, including a notable 19.2% energy reduction and a 9.0% memory reduction on the CIFAR-10 task, but at the cost of a significant 13.23 percentage point decrease in accuracy.

Table 7: Relative Performance and Efficiency Summary vs. Fair BP Baselines (Mean over 3 runs).

Algorithm Variant	Dataset	Architecture	Δ Acc. (%)	Δ Time (%)	Δ Energy (%)	Δ Mem. (%)
FF-AdamW	F-MNIST	MLP 4×2000	+0.75	+1233.26	+862.35	+1.88
FF-AdamW	MNIST	MLP 3×1000	+0.24	+342.11	+339.95	+0.85
FF-SGD	MNIST	MLP 3×1000	+0.14	+429.69	+419.67	+1.06
FF-AdamW	MNIST	MLP 4×2000	-0.01	+305.08	+236.88	+1.88
CaFo-Rand-CE	MNIST	CNN 3-block	-0.32	+88.48	+32.65	-6.67
CaFo-DFA-CE	MNIST	CNN 3-block	+0.08	+124.82	+81.83	+1.11
CaFo-Rand-CE	F-MNIST	CNN 3-block	+1.11	+205.00	+115.99	-6.67
CaFo-DFA-CE	F-MNIST	CNN 3-block	+2.47	+206.51	+201.43	+1.11
CaFo-Rand-CE	CIFAR-10	CNN 3-block	-13.23	-2.96	-19.24	-8.98
CaFo-DFA-CE	CIFAR-10	CNN 3-block	-1.72	+287.17	+301.94	+1.26
CaFo-Rand-CE	CIFAR-100	CNN 3-block	-11.44	+246.27	+188.87	-5.02
CaFo-DFA-CE	CIFAR-100	CNN 3-block	-4.43	+557.19	+576.82	+2.51
MF	MNIST	MLP 2×1000	+0.09	-12.07	-13.34	+0.86
MF	F-MNIST	MLP 2×1000	+0.51	+18.20	+9.90	+0.86
MF	CIFAR-10	MLP 3×2000	+1.21	-33.81	-40.78	-5.41
MF	CIFAR-100	MLP 3×2000	+0.37	-1.28	-12.48	-4.19

The metric Δ represents the difference between the alternative algorithm and its fair BP baseline. A positive Δ Acc. is better. Negative values for other metrics are better. The **bold** values highlight the best in class improvement for each metric across all experiments and algorithms.

In contrast, the CaFo-DFA-CE variant substantially improved accuracy, even surpassing BP on less complex datasets, yet the resource-intensive DFA pre-training phase led to profoundly greater time and energy consumption. Lastly, the Mono-Forward algorithm, when evaluated on its native MLP architectures, consistently presented the most compelling profile among the investigated alternatives. Not only did it achieve accuracy parity or superiority over its BP baselines, but it also delivered substantial reductions in training time and energy consumption, particularly on more complex datasets. The peak memory savings, while more modest, were consistent for the larger MLP architectures.

In aggregate, the experimental findings of this chapter chart an evolutionary progression from a conceptually significant but practically inefficient algorithm (FF), through a structured method defined by acute trade-offs (CaFo), to a highly promising algorithm (MF) that, within the domain of MLP architectures, effectively synthesizes superior accuracy with superior training efficiency. This empirical evidence provides the foundation for the critical analysis of algorithmic mechanisms and practical implications presented in the subsequent chapter.

5 Discussion

This chapter synthesizes the experimental results from Chapter 4, critically evaluating the performance and efficiency profiles of the alternative algorithms against their respective BP baselines. The analysis focuses on interpreting the observed trade-offs, positioning the findings within the existing literature, and assessing the practical implications of these alternative training paradigms, with a particular focus on energy efficiency. The evolutionary narrative of BP-free methods, progressing from the FF’s foundational demonstration through CaFo’s structured design to the emergence of MF as a highly efficient and performant alternative, provides the guiding framework for this discussion.

5.1 Interpretation of Key Findings

The empirical results presented in Chapter 4 provide a nuanced perspective on the capabilities of BP-free algorithms, tracing an evolutionary path toward the dual objectives of competitive accuracy and superior efficiency.

Geoffrey Hinton’s FF algorithm established a crucial proof of concept by validating that deep networks can attain competitive accuracy without a backward pass (Table 1). However, this validation was performed at a prohibitive efficiency cost. As illustrated in Figure 1, FF’s convergence process is markedly slower and more volatile than that of backpropagation, culminating in significantly extended training durations and elevated energy consumption. The analysis at the hardware level (Figure 2) critically revealed that FF’s computational patterns result in suboptimal GPU utilization and that its presumed memory advantages did not materialize for the MLP architectures tested. This positions FF as a **conceptually significant yet practically unviable baseline**, thereby motivating the investigation of more sophisticated methods.

The CaFo algorithm signifies a structured advancement, designed to deliver a more direct supervised signal via its block-wise predictor architecture. The experimental findings revealed a clear trade-off between its two evaluated variants. CaFo-Rand-CE, which uses fixed random blocks, offered consistent, though modest, peak memory savings, but incurred a substantial accuracy penalty on complex datasets (Table 3). This result highlights that effective learning cannot be achieved simply by adding local classifiers to random feature projections. In contrast, CaFo-DFA-CE substantially improved accuracy by pre-training its blocks with DFA, approaching BP’s performance. However, this success was accompanied by a considerable computational burden, as the DFA pre-training phase led to profoundly higher energy and time consumption (Table 4). CaFo consequently presents a challenging decision: one must either accept a significant performance deficit for a niche efficiency gain or incur a heavy computational cost to achieve competitive accuracy.

The MF algorithm, the most advanced method investigated, emerges from these experiments as a particularly compelling alternative for its native MLP architectures. As presented in Table 5, MF **consistently achieved or exceeded the accuracy of rigorously optimized BP baselines**. This enhanced generalization performance is attributed to its sequence of local optimizations converging to a more advantageous final validation loss compared to BP’s global optimization strategy (Figure 4). This superior performance coincided with profound efficiency improvements. On the CIFAR-10 MLP task, MF proved to be approximately 34% faster and consumed approximately 41% less energy than its BP counterpart (Table 6). This efficiency is a direct consequence of its computationally lighter design, which eliminates the backward pass. MF thus effectively navigates the trade-off between accuracy and efficiency, **substantially fulfilling the promise of a practical, high-performance, and sustainable BP-free algorithm for MLPs**.

A critical methodological element supporting these interpretations is the consistent application of systematic hyperparameter Optuna optimization and validation-based early stopping.

This protocol ensured that every algorithm was evaluated at its optimal performance point, facilitating a fair comparison of their intrinsic characteristics.

5.2 Algorithm and Architecture Strengths and Weaknesses

Each algorithm demonstrated distinct strengths and weaknesses that were frequently related to its unique learning mechanism and its interaction with the native network architecture. The Forward-Forward (FF) algorithm’s primary strength lies in its conceptual contribution: It fundamentally established that BP-free local learning can achieve competitive accuracy. However, its weaknesses are profound from a practical point of view, including a high inefficiency in both time and energy.

The Cascaded-Forward (CaFo) algorithm presents a more complex set of trade-offs. Its main weakness stems from a crucial principle revealed by these results: **the quality of feature representations is paramount in BP-free learning**. The mere attachment of local predictors, as in the Rand-CE variant, is insufficient for complex tasks. Conversely, while the DFA-CE variant produces effective features, its pre-training stage is exceptionally resource-intensive, negating any overall energy efficiency.

Mono-Forward (MF) on MLPs demonstrated the most compelling balance of strengths. It consistently achieved accuracy on par with or superior to fair BP baselines, coupled with significant reductions in training time and energy consumption. However, a critical and counterintuitive weakness emerged from the analysis: **The theoretical memory savings of BP-free learning did not fully materialize**. The peak memory advantage was modest or non-existent on the tested MLPs. This empirically demonstrates that the practical memory overhead of MF’s unique components, specifically the learnable projection matrices and their optimizer states, **can partially or fully counteract the gains** from forgoing activation storage, challenging a common assumption about BP-free methods.

Finally, backpropagation (BP) serves as a benchmark for a highly mature, robust, and generally efficient algorithm, benefiting from decades of co-optimization. Its well-known weakness is the inherent memory and computational cost of the backward pass. This analysis highlights that the optimal approach is context-dependent, with MF exhibiting a particularly strong profile for applications based on MLPs.

5.3 Practical Implications

The results of this paper carry several practical implications for the development and deployment of deep learning models, particularly concerning sustainability. The most direct consequence is the potential for significant energy savings and a reduced environmental impact. By delivering substantial reductions in training time and energy consumption while enhancing accuracy, the MF algorithm **presents a concrete path toward more sustainable AI**, translating directly into a lower estimated CO₂e. These efficiency gains also make such algorithms highly advantageous for deployment on edge devices or in other resource-constrained environments. The emergence of effective BP-free algorithms like MF **could also inspire new hardware architectures** optimized for their specific computational patterns, potentially altering trade-offs in future chip designs. Ultimately, this research provides empirical data to inform algorithm selection. For tasks based on MLPs, **MF appears to be a better choice than BP when both accuracy and efficiency are primary considerations**, suggesting that this direct local loss paradigm is a powerful direction for future algorithm development.

5.4 Theoretical Insights

This comparative investigation also yields several theoretical insights into the mechanisms of deep learning. The results robustly demonstrate that **effective high-performance learning can be realized using purely local rules**, a foundational premise established by FF and compellingly validated by MF. The success of MF’s mechanism suggests that complex credit assignment can be managed without global error propagation. By eschewing the precise symmetric weight transport of BP, these algorithms align more closely with the principles of biological learning. A key related insight is the strong coupling between the algorithm and its architecture, suggesting that **future progress in BP-free learning will likely depend on the co-design of both the learning rule and the network primitives** upon which it operates. Finally, MF’s ability to converge to a lower final validation loss than BP on the CIFAR-10 MLP task (Figure 4) offers a fascinating insight into optimization landscapes. It suggests that a sequence of greedy local optimizations may guide a model to a more advantageous region of the parameter space, **challenging the conventional wisdom that only global optimization can locate the best solutions**.

5.5 Limitations and Critical Evaluation

Although this paper provides a rigorous comparison under controlled conditions, several limitations must be acknowledged. First, the author intentionally focused the experimental scope on the *native* architectures for each algorithm. Consequently, the **strong performance of MF on MLPs does not automatically guarantee similar success on different architectures**, such as Transformers or deeper CNNs, without further research.

A second set of limitations arises from algorithm-specific nuances. The inefficiency of FF makes its current formulation impractical. For CaFo, the DFA-CE variant proved too resource-intensive to be "efficient", while the Rand-CE variant’s accuracy remains a significant weakness. Most importantly, although highly successful on MLPs, **MF’s peak memory savings were modest**, a critical nuance highlighting that the overhead of its unique components can counteract a significant portion of the gains from eliminating the backward pass.

Furthermore, while systematic hyperparameter optimization was conducted, such searches are not exhaustive. The findings are also tied to a specific hardware and software stack, and the estimated CO2e depends on the local grid’s carbon intensity. Finally, this paper focuses primarily on training efficiency; a detailed analysis of inference performance was beyond its primary scope. Despite these limitations, this work contributes a detailed and fair empirical comparison to the field of BP-free deep learning, particularly by highlighting the potential of MF as a practical, high-performance, and energy-efficient algorithm for applications based on MLPs.

6 Conclusion

This paper presented a rigorous investigation into the performance and energy efficiency of three BP-free training algorithms: FF, CaFo, and MF, benchmarked against standard backpropagation. Through a framework of fair comparisons on native architectures, incorporating direct hardware measurements, systematic hyperparameter optimization, and validation-based early stopping, this work clarifies the practical viability of these alternatives for more sustainable deep learning. The evolutionary trajectory from FF’s foundational proof of concept, through CaFo’s structured design, to MF’s emergence as a highly efficient and performant method for MLPs highlights the significant potential and rapid advancement within this domain.

6.1 Key Findings and Answers to Research Questions

The experimental results provide definitive answers to the research questions posed at the beginning of this work. Regarding classification performance, the MF algorithm, in its native MLP architectures, **robustly and consistently matched or surpassed the accuracy of its BP baseline**. Its convergence to a lower final validation loss indicates that its layer-wise optimization strategy identified a more effective solution. In contrast, FF achieved comparable accuracy but with markedly slower and more volatile convergence, while the CaFo-Rand-CE variant revealed a clear performance deficit on complex datasets. The CaFo-DFA-CE variant dramatically narrowed this gap, but at a significant computational cost during pre-training.

In terms of direct energy consumption, the MF algorithm yielded significant savings, consuming approximately 41% less energy than BP on the CIFAR-10 MLP task while achieving superior accuracy. This is in stark contrast to the case for FF, which consumed substantially more energy because of its inefficiency, and CaFo-DFA-CE, which was also more energy-intensive. This research also **empirically refutes the generalized assumption that all BP-free methods are inherently more memory efficient**. No algorithm yielded dramatic memory savings; for MF on larger MLPs, the peak memory advantage was modest (4% to 5%), as the overhead for its additional projection matrices was found to partially offset the theoretical gains.

These results clarify the different energy-performance trade-offs. FF offers an unfavorable trade-off of competitive accuracy for prohibitive inefficiency. CaFo presents a stark choice between the Rand-CE variant’s modest efficiency gains for a significant accuracy drop, and the DFA-CE variant’s competitive accuracy for a substantial efficiency cost. Crucially, MF delivered the most advantageous trade-off, achieving **accuracy parity or superiority with simultaneous and significant improvements in training time and energy efficiency**. It also demonstrated effective scalability in both performance and efficiency across the tested MLP and dataset complexities. In summary, this research established FF as a foundational proof of concept, identified CaFo as a structured method defined by clear trade-offs, and distinguished MF as a highly promising algorithm that, for MLP architectures, fulfills the objectives of achieving both BP-competitive accuracy and superior training efficiency.

6.2 Summary of Contributions

This paper makes several key contributions to the study of energy-efficient deep learning. First, it established a **rigorous and fair benchmarking framework** by replicating the native architectures for FF, CaFo, and MF, creating identical BP baselines, and applying systematic, validation-based optimization to all methods. This work presented one of the first **extensive independent evaluations of the recently proposed MF algorithm** on its native MLP architectures, robustly demonstrating its ability to improve both accuracy and efficiency. Through detailed profiling centered on **direct measurements of GPU energy consumption via NVML**, this research provided critical insights into the inefficient hardware utilization of FF and the nuanced memory characteristics of MF.

This research systematically detailed the trade-offs for each algorithm, clarifying that MF (on MLPs) provided the most favorable balance. Finally, to promote reproducibility and facilitate future research, all developed code, structured configurations, experimental protocols, and measurement tools were organized into an open research infrastructure and made publicly available.

6.3 Future Research Directions

The findings of this paper suggest several promising avenues for future research. A primary direction is architectural exploration, particularly investigating **MF’s adaptability and performance on more complex architectures** like CNNs and Transformers, which will require adapting its projection matrix mechanism. For CaFo, developing more computationally efficient, BP-free methods for training its neural blocks is essential. Further research should also evaluate the most promising algorithms on larger-scale datasets like ImageNet to fully ascertain their scalability. Other important frontiers include developing custom hyperparameter pruners for layer-wise training, deepening the theoretical understanding of how MF’s local optimization can achieve superior global solutions, and pursuing **hardware co-design for BP-free algorithms** to unlock additional energy savings. Extending these methods beyond classification to domains like generative modeling and reinforcement learning, as well as investigating their robustness and uncertainty calibration, will be crucial for their broader adoption.

6.4 Broader Implications

The research detailed in this paper has broad implications for the field. As artificial intelligence becomes increasingly prevalent, the energy consumption of these systems has become a significant social concern. The development and adoption of algorithms such as MF, which can significantly reduce the footprint of training energy without a performance penalty, represent **critical steps toward a more sustainable future for AI**. Lowering energy requirements can also **democratize AI development**, making it more accessible to researchers with limited computational resources, and is paramount for enabling powerful on-device learning under strict power constraints. The biological inspiration underlying these algorithms fosters a symbiotic relationship between AI and neuroscience, where brain-inspired computation can inform more efficient AI. Ultimately, this research contributes to a paradigm shift in which efficiency is treated as a primary design criterion, on par with accuracy. The journey beyond backpropagation is not just about devising novel algorithms; it is about **shaping a more efficient, accessible, and responsible future for artificial intelligence**.

Acknowledgments

The authors would like to thank Professor Witold Dzwinel for his invaluable guidance and expert supervision. This work was made possible by the computational resources provided by ACK Cyfronet AGH on the Athena cluster. The research was supported by the PLGrid Infrastructure.

References

- [1] Balogun Barnabas Friday and Dhakir Abbas Ali. Balancing innovation and sustainability: Assessing the impact of generative AI on energy consumption. *International Journal of Innovative Research & Development*, 13(9):1–10, 2024.
- [2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 2623–2631. ACM, 2019.
- [3] Anders S. G. Andrae and Tomas Edler. On global electricity usage of communication technology: trends to 2030. *Challenges*, 6(1):117–157, 2015.
- [4] Lasse F. Anthony, Benjamin Kanding, and Raghavendra Selvan. CarbonTracker: Tracking and predicting the carbon footprint of training deep learning models. In *ICML 2020 Workshop on Challenges in Deploying and Monitoring Machine Learning Systems*, 2020.
- [5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [6] Yoshua Bengio, Dong-Hyun Lee, Jörg Bornschein, Thomas Mesnard, and Zhouhan Lin. Towards biologically plausible deep learning. *arXiv preprint arXiv:1502.04156*, 2015.
- [7] James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyperparameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pages 2546–2554. Curran Associates, Inc., 2011.
- [8] Guo-qiang Bi and Mu-ming Poo. Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *The Journal of Neuroscience*, 18(24):10464–10472, 1998.
- [9] Bertrand Charpentier, Loris Dutto, Nicolas Thome, and Matthieu Cord. Accuracy is not the only metric that matters: Estimating the energy consumption of deep learning models using RAPL. *arXiv preprint arXiv:2304.00897*, 2023.
- [10] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [11] Jae-Won Chung, Myeongjae Lee, and Minsoo Rhu. Zeus: Understanding and optimizing GPU energy consumption of DL training. PyTorch Blog, January 2024.
- [12] Mila, BCG GAMMA, Haverford College, and Comet. CodeCarbon: Track and reduce CO2 emissions from your computing, 2021. URL: <https://github.com/mlco2/codecarbon>.
- [13] Radosvet Desislavov, Djordje Lazovic, Jie Zhou, and Pauline Grosso. Measuring the energy consumption and efficiency of deep neural networks: A survey. *arXiv preprint arXiv:2403.08151*, 2024.

- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.
- [15] Eva García-Martín, Crefeda Faviola Rodrigues, Graham Riley, and Sebastian Gabel. Estimation of energy consumption in machine learning. *Journal of Parallel and Distributed Computing*, 134:75–88, 2019.
- [16] James Gong, Bruce Li, and Waleed Abdulla. Mono-Forward: Backpropagation-free algorithm for efficient neural network training harnessing local errors. *arXiv preprint arXiv:2501.09238*, 2025.
- [17] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT Press, 2016.
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.
- [19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [20] Peter Henderson, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research*, 21(248):1–43, 2020.
- [21] Geoffrey Hinton. The Forward-Forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.
- [22] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies. In *A Field Guide to Dynamical Recurrent Neural Networks*, pages 237–243. IEEE Press, 2001.
- [23] Mark Horowitz. Computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 1.1.1–1.1.7. IEEE, 2014.
- [24] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [25] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.
- [26] Max Jaderberg, Wojciech Marian Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Decoupled neural interfaces using synthetic gradients. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1627–1635. PMLR, 2017.
- [27] Rie Johnson and Tong Zhang. Inconsistency, instability, and generalization gap of deep neural network training. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 15127–15143. Curran Associates, Inc., 2023.

- [28] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [29] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2012.
- [30] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 1097–1105. Curran Associates, Inc., 2012.
- [31] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [32] Timothy P. Lillicrap, Adam Santoro, Luke Marris, Colin J. Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.
- [33] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [34] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [35] Shreyank Narayanagowda, Animesh Laddha, Spandan Mukhopadhyay, and Pria Grover. Watt for what: Rethinking deep learning’s energy-performance relationship. *arXiv preprint arXiv:2310.06522*, 2023.
- [36] Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, pages 1037–1045. Curran Associates, Inc., 2016.
- [37] Rahmatallah Poudineh. Global electricity demand: what’s driving growth and why it matters? OIES Energy Comment, Oxford Institute for Energy Studies, January 2025.
- [38] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8026–8037, 2019.
- [39] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David R. So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.
- [40] Saurabhsingh Rajput, Yeshe Xu, Shiping Chen, and Xiaoli Jin. Enhancing energy-awareness in deep learning through fine-grained energy measurement. *arXiv preprint arXiv:2308.12264*, 2023.
- [41] Tim Yarally, Luís Cruz, Daniel Feitosa, Pauline Grosso, Djordje Lazovic, Jean-Baptiste Tallon, Matthijs van der Vlag, and Jie Zhou. Uncovering energy-efficient practices in deep learning training: Preliminary steps towards green AI. In *2023 IEEE/ACM 2nd International Conference on AI Engineering (CAIN)*, pages 127–138. IEEE, 2023.
- [42] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

- [43] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. Green AI. *Communications of the ACM*, 63(12):54–63, 2020.
- [44] Sen Song, Kenneth D. Miller, and L. F. Abbott. Competitive hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, 3(9):919–926, 2000.
- [45] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3645–3650, 2019.
- [46] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR, 2019.
- [47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008. Curran Associates, Inc., 2017.
- [48] World Economic Forum and Accenture. Artificial intelligence’s energy paradox: Balancing challenges and opportunities. AI Governance Alliance Report, World Economic Forum, January 2025.
- [49] Ginelle Greene-Dewasmes. AI and energy: Will ai reduce emissions or increase power demand? World Economic Forum, July 2024.
- [50] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- [51] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.
- [52] Gongpei Zhao, Tao Wang, Yidong Li, Yi Jin, Congyan Lang, and Haibin Ling. The Cascaded Forward algorithm for neural network training. *arXiv preprint arXiv:2303.09728*, 2023.

A Hyperparameter Tuning Details

This appendix provides a comprehensive account of the hyperparameter settings established for each algorithm in this thesis. To ensure a rigorous and fair comparison, systematic hyperparameter optimization was conducted for all training algorithms. This included Backpropagation (BP), Forward-Forward (FF), Cascaded-Forward (CaFo), and Mono-Forward (MF), each tested across their respective experimental configurations. The process was facilitated by the Optuna framework [2], a tool the author employed to methodically search for optimal parameters.

The general optimization strategy utilized a Tree-structured Parzen Estimator (TPE) sampler [7]. For the BP baselines, a Median pruner was implemented to terminate unpromising trials based on intermediate validation performance. Conversely, pruning was disabled for the alternative algorithms (FF, CaFo, MF), as their distinct, often layer-wise, training dynamics render intermediate metrics unreliable for predicting final outcomes. The primary objective for each tuning trial was the maximization of validation set accuracy, evaluated within a specified training budget. Upon completion of each search, the identified optimal hyperparameters were automatically propagated to the final experiment configuration files using dedicated utility scripts. The resulting optimized parameters, which were used for the final experimental runs detailed in Chapter 4, are documented in the following sections.

A.1 Backpropagation Baseline Tuning Parameters

The AdamW optimizer [33] was utilized for all BP baseline experiments. The optimization process was configured as follows:

- **Search Space (Common):**
 - Learning Rate (`lr`): $\text{Loguniform}(10^{-5}, 10^{-2})$
 - Weight Decay (`weight_decay`): $\text{Loguniform}(10^{-6}, 10^{-3})$
- **Objective:** Maximize validation accuracy within the trial’s training budget, which incorporated early stopping based on validation loss.
- **Sampler/Pruner:** TPE / Median.

The optimal hyperparameters identified for each BP baseline configuration were:

- **BP Baseline for FF (Fashion-MNIST 4x2000 MLP):**
 - Learning Rate: 0.0002040628171913876
 - Weight Decay: 4.186646492145325e-06
- **BP Baseline for CaFo (Fashion-MNIST 3-Block CNN):**
 - Learning Rate: 0.000564093098959669
 - Weight Decay: 8.597071465954117e-06
- **BP Baseline for CaFo (CIFAR-10 3-Block CNN):**
 - Learning Rate: 0.0067628739201322655
 - Weight Decay: 4.7418070637518205e-06
- **BP Baseline for CaFo (CIFAR-100 3-Block CNN):**
 - Learning Rate: 0.0003201243830098061
 - Weight Decay: 5.880350745731698e-05

- **BP Baseline for MF (Fashion-MNIST 2x1000 MLP):**

- Learning Rate: 0.00021316290979390737
- Weight Decay: 2.7122805865833542e-05

- **BP Baseline for MF (CIFAR-10 3x2000 MLP):**

- Learning Rate: 8.11587582031004e-05
- Weight Decay: 2.0847447927718932e-05

- **BP Baseline for MF (CIFAR-100 3x2000 MLP):**

- Learning Rate: 0.00010362161015212055
- Weight Decay: 9.105881789069153e-06

- **BP Baseline for FF (MNIST 3x1000 MLP):**

- Learning Rate: 0.0001329291894316216
- Weight Decay: 0.0007114476009343421

- **BP Baseline for FF (MNIST 4x2000 MLP):**

- Learning Rate: 0.0001329291894316216
- Weight Decay: 0.0007114476009343421

- **BP Baseline for CaFo (MNIST 3-Block CNN):**

- Learning Rate: 0.001570297088405539
- Weight Decay: 6.251373574521755e-05

- **BP Baseline for MF (MNIST 2x1000 MLP):**

- Learning Rate: 0.00019762189340280086
- Weight Decay: 7.4763120622522945e-06

A.2 Alternative Algorithm Tuning Parameters

The alternative algorithms were tuned using their respective dedicated Optuna objective functions.

- **Objective:** Maximize final validation accuracy at the end of the algorithm’s specific training procedure.
- **Sampler/Pruner:** TPE / None.

Forward-Forward (FF):

- **Search Space (Common):**

- FF Layer Learning Rate (`ff_learning_rate`): Loguniform (2×10^{-4} , 5×10^{-3})
- FF Layer Weight Decay (`ff_weight_decay`): Loguniform (10^{-4} , 10^{-3})
- Downstream Classifier Learning Rate (`downstream_learning_rate`): Loguniform (2×10^{-3} , 5×10^{-2})
- Downstream Classifier Weight Decay (`downstream_weight_decay`): Loguniform (10^{-3} , 10^{-2})

- **Best Hyperparameters Found:**

- FF (Fashion-MNIST 4x2000 MLP, AdamW):
 - * `ff_learning_rate`: 0.0003722230492626773
 - * `ff_weight_decay`: 0.0003583569744251919
 - * `downstream_learning_rate`: 0.011120232890144957
 - * `downstream_weight_decay`: 0.0051200359925764015
- FF (MNIST 3x1000 MLP, SGD):
 - * `ff_learning_rate`: 0.0019075980272792064
 - * `ff_weight_decay`: 0.0005229088821652552
 - * `downstream_learning_rate`: 0.03234998392840941
 - * `downstream_weight_decay`: 0.0047668064095875455
 - * `ff_momentum`: 0.9
 - * `downstream_momentum`: 0.9
- FF (MNIST 3x1000 MLP, AdamW):
 - * `ff_learning_rate`: 0.0004759134394296565
 - * `ff_weight_decay`: 0.0004169096909947932
 - * `downstream_learning_rate`: 0.010727571797342173
 - * `downstream_weight_decay`: 0.0061527185182837065
- FF (MNIST 4x2000 MLP, AdamW):
 - * `ff_learning_rate`: 0.0005253511034001832
 - * `ff_weight_decay`: 0.0004227844165066995
 - * `downstream_learning_rate`: 0.0027075510107757122
 - * `downstream_weight_decay`: 0.0035730744916099627

- **Fixed Parameters:** Key parameters were held constant during the tuning of the FF algorithm. These included the optimizer type (AdamW or SGD), a goodness threshold ($\theta = 2.0$), and peer normalization settings (factor=0.03, momentum=0.9). All trials were conducted for a maximum of 100 epochs, with early stopping triggered by the FF_Hinton/Val_Acc_Epoch metric, using a patience of 20 epochs and a minimum delta of 0.01.

Cascaded-Forward (CaFo):

- **Search Space (Common):**

- Predictor Learning Rate (`predictor_lr`): Loguniform (2×10^{-4} , 5×10^{-3})
- Predictor Weight Decay (`predictor_weight_decay`): Loguniform (10^{-7} , 10^{-4})
- Epochs Per Predictor (`num_epochs_per_block`): IntUniform (range dependent on dataset)
- For CaFo-DFA variant (*`train_blocks=true`*):
 - * Block Learning Rate (`block_lr`): Loguniform (2×10^{-5} , 5×10^{-4})
 - * Block Weight Decay (`block_weight_decay`): Loguniform (10^{-7} , 10^{-4})

- **Best Hyperparameters Found:**

- CaFo-Rand-CE (Fashion-MNIST 3-Block CNN):
 - * `predictor_lr`: 0.0013737845955327983

- * predictor_weight_decay: 2.937538457632828e-07
 - * num_epochs_per_block: 145 (Maximum, with early stopping patience of 8)
- CaFo-DFA-CE (Fashion-MNIST 3-Block CNN):
 - * block_lr: 0.0001384690524742107
 - * block_weight_decay: 1.3311216080736884e-05
 - * predictor_lr: 0.0003304463634024569
 - * predictor_weight_decay: 3.967605077052991e-05
 - * num_epochs_per_block: 85 (Maximum, with early stopping patience of 6)
- CaFo-Rand-CE (CIFAR-10 3-Block CNN):
 - * predictor_lr: 0.00024111676098423975
 - * predictor_weight_decay: 6.3583588566762514e-06
 - * num_epochs_per_block: 733 (Maximum, with early stopping patience of 8)
- CaFo-DFA-CE (CIFAR-10 3-Block CNN):
 - * block_lr: 0.0001373784595532798
 - * block_weight_decay: 2.938027938703532e-07
 - * predictor_lr: 0.0006677511008261821
 - * predictor_weight_decay: 1.5702970884055385e-05
 - * num_epochs_per_block: 147 (Maximum, with early stopping patience of 8)
- CaFo-Rand-CE (CIFAR-100 3-Block CNN):
 - * predictor_lr: 0.00024111676098423975
 - * predictor_weight_decay: 6.3583588566762514e-06
 - * num_epochs_per_block: 1220 (Maximum, with early stopping patience of 10)
- CaFo-DFA-CE (CIFAR-100 3-Block CNN):
 - * block_lr: 0.0001373784595532798
 - * block_weight_decay: 2.938027938703532e-07
 - * predictor_lr: 0.0006677511008261821
 - * predictor_weight_decay: 1.5702970884055385e-05
 - * num_epochs_per_block: 1271 (Maximum, with early stopping patience of 10)
- CaFo-Rand-CE (MNIST 3-Block CNN):
 - * predictor_lr: 0.00042136234742798395
 - * predictor_weight_decay: 1.6585525961391826e-06
 - * num_epochs_per_block: 227 (Maximum, with early stopping patience of 8)
- CaFo-DFA-CE (MNIST 3-Block CNN):
 - * block_lr: 0.0004977965859721583
 - * block_weight_decay: 9.594431243311431e-07
 - * predictor_lr: 0.00025391813833013565
 - * predictor_weight_decay: 2.0840357948896564e-06
 - * num_epochs_per_block: 12 (Maximum, with early stopping patience of 5)
- **Fixed Parameters:** Several parameters for the CaFo algorithm remained fixed. The block architecture was a three-block CNN, with each block comprising a 3x3 Convolution, a ReLU activation, a 2x2 Max-Pooling layer, and Batch Normalization, with output channels of 32, 128, and 512 respectively. The Adam optimizer and Cross-Entropy loss were consistently used for predictor training, and the final predictions were aggregated by summation.

Mono-Forward (MF):

- **Search Space (Common):**

- Learning Rate (`lr`): Loguniform (10^{-5} , 10^{-2})
- Epochs Per Layer (`epochs_per_layer`): IntUniform (e.g., 5-30)

- **Best Hyperparameters Found:**

- MF (MNIST 2x1000 MLP):
 - * Learning Rate: 0.001570297088405539
 - * Epochs Per Layer: 14
- MF (Fashion-MNIST 2x1000 MLP):
 - * Learning Rate: 0.0003873086262136253
 - * Epochs Per Layer: 12
- MF (CIFAR-10 3x2000 MLP):
 - * Learning Rate: 0.0003241756767272183
 - * Epochs Per Layer: 15
- MF (CIFAR-100 3x2000 MLP):
 - * Learning Rate: 0.00017277890583771544
 - * Epochs Per Layer: 8

- **Fixed Parameters:** For the MF algorithm, several key parameters were held constant to ensure consistency. The MLP architecture was fixed as specified for each dataset: a 2x1000 MLP for MNIST and Fashion-MNIST, and a 3x2000 MLP for CIFAR-10 and CIFAR-100. Layer-wise training was conducted using the Adam optimizer with a weight decay value fixed at 0.0, minimizing a local Cross-Entropy loss at each layer. All layer weights and their corresponding projection matrices were initialized using the Kaiming Uniform method.

B Computational Environment

The experiments presented in this thesis were conducted within a consistent and well-defined computational environment hosted on the Athena cluster. The specific hardware and software configurations are detailed below to ensure the reproducibility of the presented results.

- **Operating System:** Rocky Linux 9.5 (Blue Onyx)
- **CPU Architecture:** x86_64 (AMD EPYC 7742 @ 2.25 GHz)
- **GPU Architecture:** NVIDIA Ampere (A100-SXM4-40GB)
- **NVIDIA Driver Version:** 570.86.15
- **CUDA Toolkit Version:** 12.4.0
- **Deep Learning Framework:** PyTorch v2.4.0 (build ‘+cu121’ linked against CUDA 12.1)
- **Python Version:** 3.10.4
- **Key Python Libraries:**

– torchvision: 0.19.0	– pynvml: 12.0.0	– PyYAML: 6.0.2
– optuna: 4.2.1	– codecarbon: 3.0.1	– scikit-learn: 1.6.1
– wandb: 0.19.8	– numpy: 2.2.4	– tqdm: 4.67.1
- **Profiling Tool for FLOPs:** PyTorch Profiler (`torch.profiler`)
- **Energy and Resource Monitoring Tool:** NVIDIA Management Library (NVML) via the `pynvml` package
- **Carbon Footprint Estimation Tool:** CodeCarbon v3.0.1 [12]
- **Job Scheduler:** SLURM 23.11.7

C Code Repository and Experimental Logs

To ensure the highest degree of reproducibility and transparency, the complete source code for all experiments, along with the unabridged output logs from every run, has been made publicly available. This repository contains the full implementations of the alternative algorithms and their backpropagation baselines, all scripts for data loading and experiment execution, and the raw log files for complete verifiability of the results. The repository is hosted on GitHub at the following address: <https://github.com/Przemyslaw11/BeyondBackpropagation>

Comprehensive instructions for setting up the environment and replicating the experiments are provided in the `README.md` file within the repository. For specific software versions and hardware details, the author refers the reader to the environmental specifications listed in Appendix B.